

RailS: Load Balancing for All-to-All Communication in Distributed Mixture-of-Experts Training

Heng Xu¹, Zhiwei Yu¹, Chengze Du¹, Ying Zhou¹, Letian Li², Haojie Wang³, Weiqiang Cheng,
and Jialong Li¹, *Member, IEEE*

Abstract—Training Mixture-of-Experts (MoE) models introduces sparse and highly imbalanced all-to-all communication that dominates iteration time. Conventional load-balancing methods fail to exploit the deterministic topology of Rail architectures, leaving multi-NIC bandwidth underutilized. We present RailS, a distributed load-balancing framework that minimizes all-to-all completion time in MoE training. RailS leverages the Rail topology’s symmetry to prove that uniform sending ensures uniform receiving, transforming global coordination into local scheduling. Each node independently executes a Longest Processing Time First (LPT) spraying scheduler to proactively balance traffic using local information. RailS activates N parallel rails for fine-grained, topology-aware multipath transmission. Across synthetic and real-world MoE workloads, RailS improves bus bandwidth by 20%–78% and reduces completion time by 17%–78%. For Mixtral workloads, it shortens iteration time by 18%–40% and achieves near-optimal load balance, fully exploiting architectural parallelism in distributed training.

Index Terms—Mixture-of-experts models, load balancing, all-to-all communication, rail-optimized architecture.

I. INTRODUCTION

ARTIFICIAL intelligence (AI) training has shifted from dense models to mixture-of-experts (MoE) models, making cross-GPU communication a critical bottleneck. While early dense models [11], [22] used structured collective communication over millions of parameters, modern large language models (LLMs) [9], [57], [62] rely on multi-level parallelism and collective operations at the scale of billions of parameters. MoE models [13], [14], [20], [27], [33], [39], [52] introduce expert parallelism with tens of billions of parameters, where all-to-all communication generates sparse and highly imbalanced traffic across hundreds of GPUs.

This all-to-all communication makes load balancing a central challenge. Studies show that such communication

dominates MoE iterations, with latency limiting distributed training throughput [38]. To mitigate this, the Rail architecture [43] employs multi-NIC direct-to-leaf links to relieve bandwidth bottlenecks in conventional spine-leaf networks. Specifically, it provides multiple parallel, deterministic physical links by binding distinct NICs to separate leaf switches to maximize aggregate throughput. Nevertheless, existing deployments remain suboptimal for MoE’s irregular traffic patterns and dynamic load-balancing needs, leaving hardware potential largely untapped.

Challenge 1: The inherent path selection conflicts in the Rail architecture can completely negate its hardware parallelism advantages.

Path selection [25], [42], [49], [63], [66], [69] critically affects communication efficiency. Rail offers two deterministic path options for all-to-all traffic, each with limitations. The first routes traffic through the spine layer. It works with traditional topologies but relies on hashing [24], [68], which ignores MoE traffic sparsity. This causes large flows to cluster on a few links, creating congestion while other paths remain idle. The second option uses NIC-direct rails. Without advanced transport support such as multipath transmission, flow splitting, reordering, and reassembly, the bottleneck stays at a single node. Spine paths fail due to architecture limits, and direct rails are underutilized because of immature transport protocols, restricting Rail’s parallelism.

Challenge 2: Existing load balancing methods overlook topological characteristics, preventing their scheduling decisions from approaching the theoretical upper bound.

Current load balancing approaches ignore Rail’s deterministic topology. ECMP and dynamic load balancers [5], [23], [31], [48], [59] treat multiple paths as interchangeable and select them at random. However, Rail has N parallel rails with a one-to-one mapping from each source NIC to a destination NIC. This predictable and symmetric structure offers prior knowledge useful for global optimization. Existing methods overlook it, so both static hashing and dynamic probing choose paths blindly, fail to exploit the topology, and cannot achieve the theoretical optimum, limiting system throughput.

Challenge 3: The granularity of traffic splitting involves an inherent trade-off, creating a dilemma between load balancing and system efficiency.

In load balancing, loss is inevitable due to the granularity of traffic splitting. Splitting traffic at the subflow level [45], [54] can lead to collisions and uneven utilization across N NIC paths, even when flows are further divided or recombined. Splitting traffic at the flowlet [28], [58], [60] or packet level [8], [12], [32] can cause excessive simultaneous arrivals at a

Received 22 October 2025; revised 9 February 2026; accepted 17 March 2026; approved by IEEE TRANSACTIONS ON NETWORKING Editor X. Luo. Date of publication 23 March 2026; date of current version 31 March 2026. This work was supported by Guangdong Provincial Young Innovative Talent Program. (Corresponding author: Jialong Li.)

Heng Xu, Chengze Du, Ying Zhou, and Jialong Li are with the Faculty of Computer Science and Artificial Intelligence, Shenzhen University of Advanced Technology, Shenzhen 518107, China (e-mail: lijialong@suat-sz.edu.cn).

Zhiwei Yu is with the Institute for Network Sciences and Cyberspace, Tsinghua University, Beijing 100084, China.

Letian Li is with the Department of Information Engineering, The Chinese University of Hong Kong, Hong Kong.

Haojie Wang and Weiqiang Cheng are with China Mobile, Beijing 100032, China.

Digital Object Identifier 10.1109/TON.2026.3676382

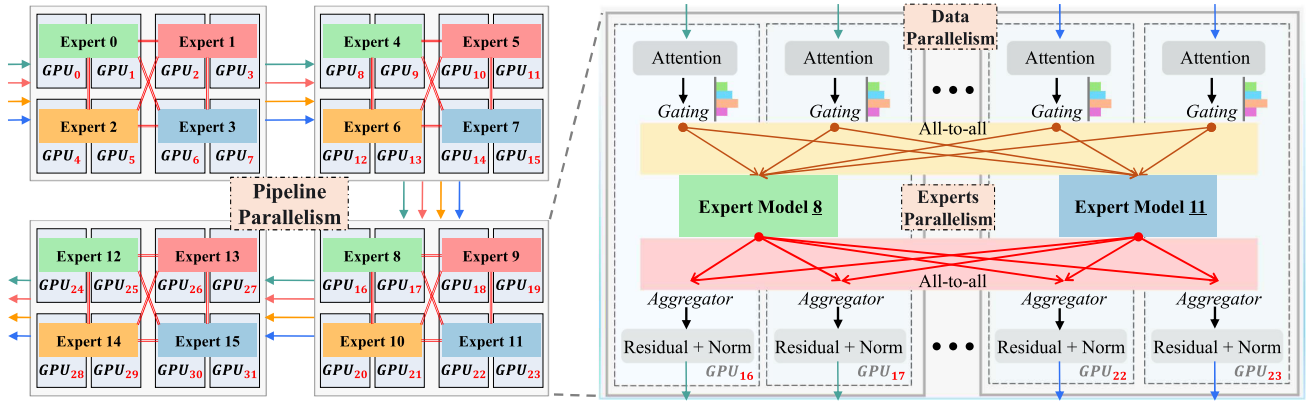


Fig. 1. Hybrid parallelism for a MoE model (data parallelism, pipeline parallelism, expert parallelism, and tensor parallelism).

shared link, overflowing buffers and resulting in packet loss throughout the network, as in all-to-all traffic patterns. Coarse splitting leads to poor load balancing, whereas overly fine splitting overwhelms NICs and incurs excessive system overheads. While finer granularity improves network utilization, it exacerbates system overheads, exposing a fundamental trade-off between load balance and hardware efficiency.

To address these challenges, we propose RailS, a distributed load balancing framework that minimizes all to all completion time. RailS systematically resolves the identified challenges through three core contributions.

First, to resolve the path selection conflicts (Challenge 1), RailS activates N parallel physical rails for fine grained transmission. Our theoretical analysis leverages Theorem 1 to show that Rail provides N parallel logical channels between any pair of nodes, quantifying its maximum aggregate bandwidth. Building on this, Theorem 2 formulates the minimization of communication time as a min max load balancing problem. These results indicate that fully bypassing spine layer forwarding, which is inefficient for MoE traffic, and exclusively using the N direct connected rails constitutes the optimal strategy. RailS operationalizes this by activating these previously under-utilized parallel paths to support dynamic multipath spraying.

Secondly, RailS addresses topological blindness (Challenge 2) by exploiting the deterministic Rail topology to guide scheduling. Theorem 3 proves a high symmetry between sending and receiving loads, establishing that any strategy maximizing uniform sending inherently guarantees uniform receiving. This theoretical foundation allows RailS to decompose global coordination into local subproblems. Consequently, we implement a distributed Longest Processing Time scheduler, where each node optimizes decisions based solely on local information while collaboratively achieving a near optimal global solution, thereby avoiding heavy synchronization overheads.

Finally, RailS resolves the granularity trade off (Challenge 3) through a theory driven traffic partitioning mechanism. To balance load distribution against system overhead, the underlying control coalescing mechanism proactively divides large flows into fixed size atomic chunks, replacing reactive packet level or flowlet level splitting. Theorem 4 proves that the load imbalance of our approximation algorithm is strictly bounded by the maximum chunk size $w_{\max}$. This provides theoretical guidance for selecting an optimal granularity that minimizes queuing delays while maintaining high hardware efficiency.

We validate RailS in a large-scale, programmable datacenter simulator, scaling to 128 compute domains with 1,024 GPUs to match the targeted cluster scale, using a variety of synthetic workloads, including uniform, sparse, and skewed patterns, as well as real-world MoE traces. RailS consistently outperforms baselines: under sparse loads, bus bandwidth improves by 20%–78% across different sparsity levels and total completion time decreases by 17%–78%; in sender-skewed and receiver-skewed scenarios, it achieves the lowest mean squared error across NICs, indicating optimal load balance. For Mixtral workloads, RailS shortens iteration time by 18%–40%. In dense setups, it reduces collective completion time by 8.9%–44.3%, and in sparse setups, by 66.1%–80.4%. Furthermore, to ensure end-to-end correctness in production scenarios, we deploy RailS on a physical RDMA testbed evaluating real MoE training workloads, confirming its efficiency for models with tens to hundreds of billions of parameters. These results demonstrate RailS’s ability to fully exploit architectural parallelism while maintaining consistent load balance across diverse and imbalanced workloads.

II. BACKGROUND AND MOTIVATION

A. Research Background

Model Communication Challenges. AI training has evolved from deep neural networks (DNNs) with millions of parameters to LLMs with tens of billions, greatly increasing communication demands. Early data-parallel DNNs relied on allreduce for gradient synchronization with limited bandwidth pressure. In contrast, LLMs employ hybrid parallelism including data, tensor, and pipeline modes, where communication volume scales with model size and coordination complexity.

Specifically, Data Parallelism (DP) replicates the model across GPUs to process different data subsets simultaneously. Tensor Parallelism (TP) splits individual layers’ matrix operations across devices to distribute memory and computation. Pipeline Parallelism (PP) partitions model layers across stages to enable concurrent execution. Finally, Expert Parallelism (EP) distributes experts across nodes, necessitating high-bandwidth all-to-all communication for token routing. Collectively, these strategies transform training into a complex distributed system, introducing intricate communication patterns that shift the primary performance bottleneck from computation to network interconnects.

In Fig. 1, MoE models further introduce sparsely activated experts, with only a subset active per sample. This

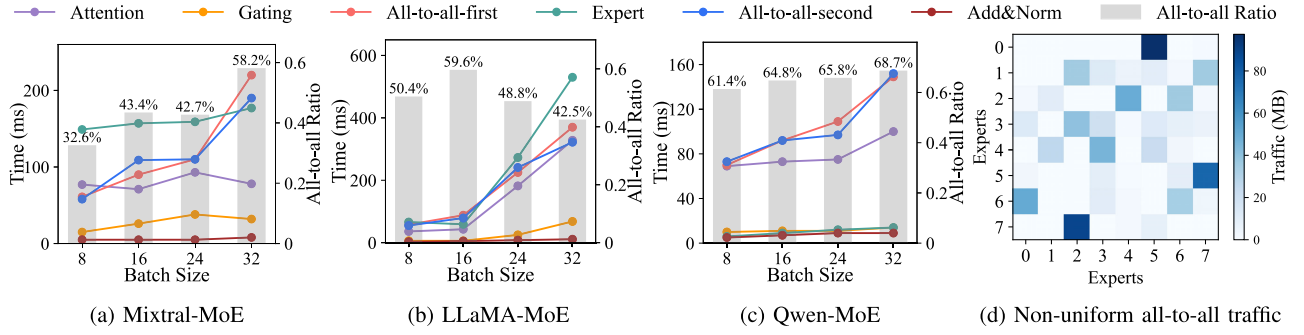


Fig. 2. Actual measured overhead [38] of different modules in MoE models under expert parallelism: (a) Mixtral $8 \times 7B$, (b) LLaMA 6.7B, and (c) Qwen 14.3B. The all-to-all ratio indicates the measured proportion of iteration time spent on all-to-all communication. (d) Empirically measured traffic matrix of Mixtral $8 \times 7B$.

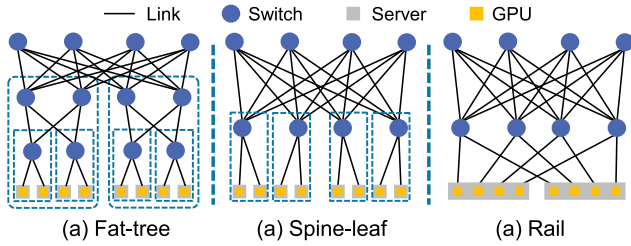


Fig. 3. Network architectures evolution.

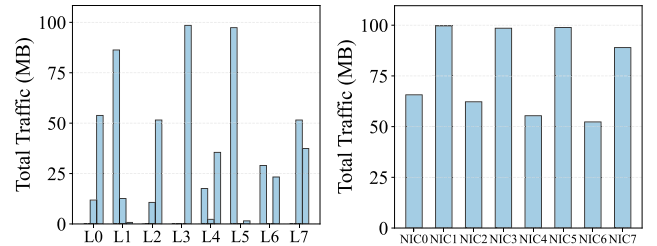
produces highly sparse and dynamic all-to-all communication in Fig. 2, where all-to-all occupies 40%–60% of the iteration time, and its load imbalance confirms the impact on performance. Consequently, MoE training reveals the bottlenecks of communication-limited distributed systems.

Network Architectures Evolution. In Fig. 3, to meet the growing bandwidth and latency demands of AI training, data center networks [3], [17], [18], [19], [43], [55] have evolved from Fat-tree to Spine-leaf and Rail architectures. Three-tier designs suffer from high latency and centralized bottlenecks, while spine-leaf architectures reduce latency over three-tier designs, they fall short under GPU-scale all-to-all workloads, causing congestion and imbalance. The Rail architecture, adopted by NVIDIA, connects multiple NICs to different leaf switches for single-hop GPU paths and parallel bandwidth use. Yet in MoE training, dynamic loads and path constraints still cause NIC congestion and bandwidth underutilization.

Limitations of Deterministic Paths. Multipath transport and load balancing are critical for improving datacenter communication and supporting coordination among distributed nodes. In clusters optimized with the Rail architecture, inter-node communication typically relies on two deterministic path options, both of which exhibit limitations under MoE workloads.

The first option is the spine-routed path, which uses ECMP to hash traffic onto uplinks toward the spine layer. While this approach resembles conventional load balancing, it ignores the sparsity of MoE traffic, causing large elephant flows to collide on specific links while others remain underutilized.

The second option is the NIC-direct path, where traffic is forwarded strictly between corresponding NICs, bypassing the spine layer. However, RDMA [53] and RoCE [1], [2] statically binds communication to a single path. This rigid mapping fails to exploit the topological parallelism offered by multiple Rails, effectively limiting subflow coordination and exacerbating bursty all-to-all traffic patterns.



(a) Uplink traffic per leaf switch. (b) NIC-direct traffic for a server.

Fig. 4. Traffic for (a) uplink traffic per leaf switch and (b) NIC-direct traffic.

We conducted a series of experiments using Mixtral $8 \times 7B$ and monitored the load distribution. As shown in Fig. 4a, sparse MoE traffic is difficult to balance effectively using ECMP. In Fig. 4b, failing to utilize multiple NICs leads to single-point bottlenecks. Consequently, existing frameworks struggle to overcome these limitations, motivating the need for a solution that can dynamically exploit multipath [51] diversity without introducing protocol complexity.

B. Research Motivation

To quantify the limitations of current deployment schemes, we conducted a characterization study on a Rail cluster consisting of 3 spine switches, 8 leaf switches, and 3 servers with a total of 24 GPUs, simulating the execution of an actual Mixtral $8 \times 7B$ workload. We evaluated ECMP, static NIC-direct routing, and spraying (a strategy that actively disperses traffic load across different network paths) with different flow-splitting granularities. The results reveal three fundamental challenges, motivating our design.

Path Selection Conflicts. Current architectures rely on two deterministic path-selection schemes, but our experiments show that neither can accommodate the sparse nature of MoE traffic. As shown in Fig. 4a, routing via ECMP over the spine network leads to severe load imbalance: the hash-based mechanism causes elephant flows to collide on specific uplinks while leaving other links underutilized. On the other hand, strictly using static Rail-direct paths creates single-point bottlenecks. In Fig. 4b, traffic hotspots on specific GPUs directly translate into network hotspots, resulting in up to $2 \times$ load imbalance. In Fig. 5a, the MoE all-to-all communication time under both schemes is significantly lower than RailS, highlighting the urgent need for a new solution.

Topological Blindness. Existing mitigation approaches, such as disabling NIC congestion control [15], [67] or

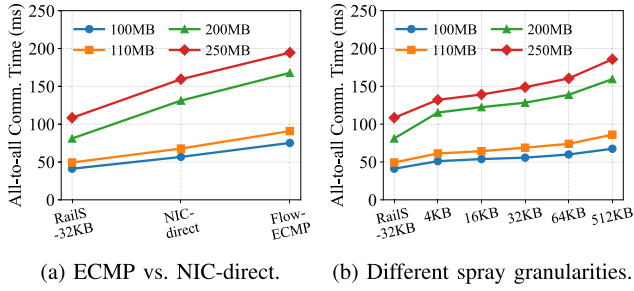


Fig. 5. MoE all-to-all communication time: (a) ECMP vs. NIC-direct path and (b) different spray granularities.

adopting dual-plane network architectures [47], suffer from deadlock risks [26], [41] or high cost, and fail to address the fundamental mismatch between topology and sparse traffic. Meanwhile, current load balancers do not fully exploit the symmetry of the Rail topology. Our experiments comparing spraying with topology-aware scheduling show that even fine-grained flow splitting does not achieve satisfactory performance. Specifically, as observed in Fig. 5b, a generic 4KB packet splitting scheme that ignores topological features underperforms the topology-aware RailS-32KB scheme. Without recognizing that uniform sending on the Rail network leads to uniform receiving, generic strategies cannot guarantee load balance under the skewed traffic patterns typical of MoE training, thereby hindering global optimization.

The Granularity Trade-off. Achieving load balance requires flow splitting, but the granularity of splitting introduces a fundamental trade-off between network efficiency and system resources. As shown in Fig. 5b, although finer granularity improves load balancing performance compared to coarse granularity, the benefits gradually diminish. Crucially, extremely fine-grained splitting incurs excessive system overhead, such as increased PCIe transactions and MMIO events [69]. In contrast, coarse-grained splitting leads to uneven load distribution. These observations highlight a key issue: designers must balance effective load balancing against hardware efficiency, rather than simply pursuing the finest possible granularity.

III. OVERVIEW

This section provides a high-level overview of the proposed traffic scheduling system. First, we describe the core design premises and problem boundaries of the system. Next, we present the overall system design blueprint, including its key mechanisms and optimization objectives. Finally, this section summarizes the critical theoretical contributions introduced in subsequent sections, providing readers with a clear roadmap.

A. Design Premises and Problem Scope

Before detailing our system design, this section first clarifies the core premises, model assumptions, and problem boundaries of our work.

Network Topology. In Fig. 6, this study focuses on the Rail network architecture, which equips each compute node with multiple NICs, each directly connected to a different leaf switch, providing a hardware foundation for multipath parallel communication. All subsequent theoretical analyses and system designs are built upon the intrinsic structural characteristics of this emerging topology.

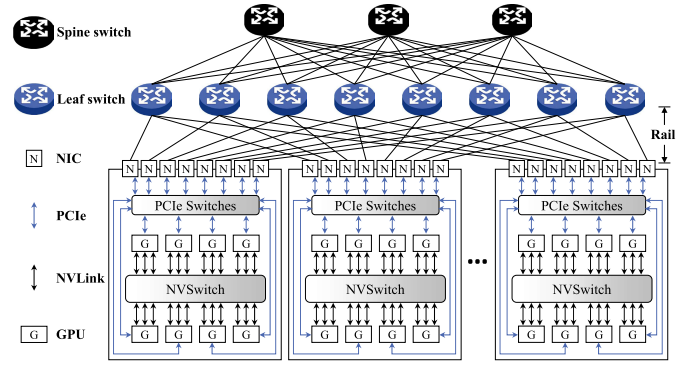


Fig. 6. Rail-optimized network topology.

Traffic Model. Our optimization target is the key performance bottleneck in current large-scale distributed training: all-to-all communication generated by MoE models. This traffic exhibits significant sparsity and imbalance, rendering traditional static load-balancing mechanisms inefficient. Our theoretical analysis and real-time scheduling are based on the known inter-GPU traffic matrix $D^{(1)}$ and inter-domain traffic matrix $D^{(2)}$.

System Abstraction. At the software level, the system adopts a scalable transport-layer framework that decouples the control and data paths of RDMA NICs. This separation enables flexible, software-defined implementation of transport-layer protocols on the host CPU. The extensibility provides the technical foundation for integrating a custom LPT scheduler to enable proactive load balancing.

B. System Design Overview

We design and propose RailS, a distributed traffic load balancer specifically optimized for MoE communication under the Rail architecture. The system aims to minimize the communication completion time T^* . Its core idea is to actively shape and schedule sparse, imbalanced flows at each sender node through a set of coordinated mechanisms, thereby approximating globally optimal load balancing.

The RailS's design comprises three core mechanisms forming an efficient splitting-scheduling-spraying pipeline:

Flow Splitting. Serving as the entry point of the pipeline, this mechanism decomposes large, coarse-grained messages generated by the upper-layer applications into a series of smaller, atomic flow units. This preprocessing step is crucial for fine-grained load balancing and for reducing the maximum flow size $w_{\max}$.

LPT Scheduling. Acting as the brain of the system and the core of the proactive spraying, this mechanism differs from traditional strategies that rely on passive feedback such as network latency. Using an LPT-based scheduler, it deterministically assigns each flow to the optimal target track based on the sizes of all pending atomic flows and the real-time local load state of each track, thereby actively constructing a highly balanced flow allocation.

Multipath Spraying. This mechanism provides the macroscopic strategy for flow distribution. It distributes the split atomic flows across all N available parallel communication tracks at each node. Unlike static-hash-based ECMP, spraying mechanism is fully dynamic and policy-driven.

Through the coordinated operation of these mechanisms, RailS effectively decomposes a complex global optimization

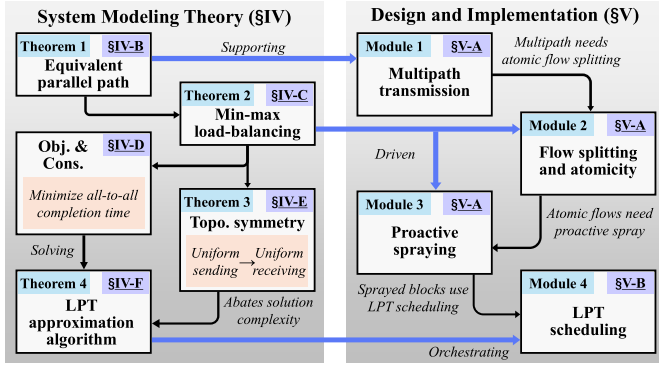


Fig. 7. Overview of system workflow.

problem into distributed local decisions that can be executed independently and efficiently at each node, ultimately translating the theoretical insights from section IV into practical system performance.

C. Core Theoretical Contributions Roadmap

This section outlines how a series of interconnected theoretical analyses establish the foundation for the subsequent system design of RailS. Fig. 7 illustrates the workflow.

Question 1 (§ IV-B): What is the Theoretical Communication Capacity of the Rail Architecture? Our analysis begins by rigorously defining the fundamental capability of the Rail network topology. Using the max-flow min-cut method, we prove that the architecture provides N parallel, equal-capacity logical channels for arbitrary inter-domain communication (Theorem 1). This result not only quantifies the system’s bandwidth upper bound but also establishes the feasibility and necessity of multipath load balancing.

Question 2 (§ IV-C): How to Formulate the Time Minimization Problem? Having established the network capacity, we reformulate the core optimization objective of minimizing all-to-all communication time into a mathematical framework. We prove that this complex dynamic time optimization problem is equivalent to a more tractable min-max load-balancing problem (Theorem 2). This equivalence provides a clear and actionable optimization objective for designing allocation strategies.

Question 3 (§ IV-E): What is the Optimal State for Load Balancing? To define an “ideal target” for practical scheduling algorithms, we first solve the problem under the idealized assumption that flows are arbitrarily divisible. We prove that the optimal strategy admits a concise closed-form solution, namely a perfectly uniform distribution across N rails (Theorem 3). This result not only reveals the intrinsic symmetry between sending and receiving loads in the Rail architecture but also sets the theoretical performance ceiling for all practical approximation algorithms.

Question 4 (§ IV-F): How to Approximate the Optimum with Discrete Flows? Finally, we address the more realistic scenario of indivisible (atomic) flows. We propose the LPT approximation algorithm and prove that it efficiently approaches the ideal uniform state established in Step 3, with a guaranteed performance bound where the mean squared error is upper bounded by $w_{\max}^2$ (Theorem 4). This provides a solid theoretical basis for implementing efficient, low-overhead local scheduling in distributed nodes.

IV. PROBLEM FORMULATION

In this section, we model the Rail-based network topology and reformulate the all-to-all communication optimization as an equivalent load-balancing problem. We derive closed-form optimal strategies for both continuous and atomic flow scenarios and propose an efficient approximation algorithm with proven performance guarantees, establishing a solid theoretical foundation for system design.

A. System Model

We consider a large-scale GPU cluster system composed of M computing domains. The system is communication-constrained and supports distributed training of LLMs and MoE models. Each domain contains N GPUs interconnected via intra-domain networks. Any GPU in the system can be uniquely identified by a tuple (d, n) , where $d \in \{1, \dots, M\}$ is the domain index and $n \in \{1, \dots, N\}$ is the GPU index within that domain. Each GPU (d, n) is connected to a dedicated network interface card, denoted as $\text{NIC}_{d,n}$.

The inter-domain communication architecture follows a Rail-based design. Specifically, for any fixed intra-domain index n , the n -th NIC of all domains (i.e., $\text{NIC}_{1,n}, \text{NIC}_{2,n}, \dots, \text{NIC}_{M,n}$) connects to the same Leaf switch S_n . Hence, the system contains N such switches, forming N parallel communication “rails.” The Spine and Leaf layers remain fully connected.

To precisely analyze this system, we define the following key concepts.

Network Link Rates. Two types of link rates are considered. The intra-domain forwarding rate R_1 represents the data exchange rate among GPUs within a single domain. The inter-domain forwarding rate R_2 corresponds to the forwarding rate of domain NICs or switches and reflects the data processing capability of switch ports. We don’t need to assume that R_1 is uniform. It only needs to be significantly higher than the inter-domain transmission rate, which is consistent with the characteristics of high-bandwidth intra-node interconnects such as NVLink and PCIe.

Traffic Matrix. To quantify communication demands, we define two levels of traffic matrices. The GPU-to-GPU traffic matrix $D^{(1)}$ has elements $D_{(d,n),(f,m)}^{(1)}$ representing the traffic from source GPU (d, n) to destination GPU (f, m) . Aggregating these yields the domain-to-domain traffic matrix $D^{(2)}$, where each element $D_{d,f}^{(2)}$ denotes the total traffic from domain d to domain f :

$$D_{d,f}^{(2)} = \sum_{n=1}^N \sum_{m=1}^N D_{(d,n),(f,m)}^{(1)} \quad (1)$$

Allocation Matrix. The allocation matrix P is a three-dimensional decision variable describing the routing of traffic. Its elements $P_{k,f,n}$ denote the proportion of total traffic from source domain k to destination domain f assigned to the n -th communication rail. The matrix and constraints are given by:

$$P = [P_{k,f,n}] \in \mathbb{R}^{M \times M \times N} \quad (2)$$

$$\text{s.t. } \sum_{n=1}^N P_{k,f,n} = 1, \quad P_{k,f,n} \geq 0, \quad \forall k, f, n \quad (3)$$

Load Matrix. Given an allocation matrix P , the communication load on each NIC can be computed. We define the

sending load matrix $\mathbf{S} \in \mathbb{R}^{M \times N}$ and receiving load matrix $\mathbf{R} \in \mathbb{R}^{M \times N}$:

$$\mathbf{S}_{k,n} = \sum_{f=1}^M D_{k,f}^{(2)} P_{k,f,n} \quad (4)$$

$$\mathbf{R}_{f,n} = \sum_{k=1}^M D_{k,f}^{(2)} P_{k,f,n} \quad (5)$$

All-to-All Completion Time. In the all-to-all communication pattern, the total completion time T is defined as the time from the start of the collective operation until all inter-domain data transfers required by the traffic matrix $D^{(2)}$ are finished. As a collective operation involving numerous parallel flows, the completion of the entire task is dictated by the slowest flow, as synchronisation barriers prevent subsequent computations until all flows arrive. Note that this time T represents the latency of a single communication phase (e.g., Dispatch or Combine) and serves as a critical component of, yet is distinct from, the full training iteration. Minimizing the time T^* is one of the main optimization objectives.

Iteration Time. To contextualize the impact of T , we define the training iteration time as the total duration to process a data batch. A single iteration comprises a forward pass and a backward pass. The forward pass typically includes attention computation, a dispatch all-to-all communication to route tokens to experts, expert computation, and a combine all-to-all communication to aggregate results. The backward pass mirrors this structure for gradient synchronization. Consequently, the all-to-all completion time T occurs multiple times per iteration, and its reduction directly shortens the overall training time.

Load Balancing Mean Squared Error. To quantify the deviation between actual NIC loads and the ideal target within a domain, we define the mean squared error (MSE). Let the ideal target load T_{opt} be the average load of the domain, and let L_j denote the load assigned to the j -th NIC. The MSE is:

$$\text{MSE} = \frac{1}{N} \sum_{j=1}^N (L_j - T_{\text{opt}})^2 \quad (6)$$

B. Inter-Domain Communication Capacity Analysis

We now analyze the fundamental limits of inter-domain communication in the Rail-based network. This section quantifies the theoretical upper bound of cross-domain throughput and establishes its equivalence-in-use to a set of parallel logical rails, which provides the analytical foundation for subsequent flow allocation and optimization.

Theorem 1: Under the defined system model and assuming that the intra-domain forwarding rate is higher than the inter-domain forwarding rate ($R_1 > R_2$), the maximum achievable unidirectional throughput between any two distinct domains k and f is:

$$\text{Cap}_{k \rightarrow f} = N \cdot R_2 \quad (7)$$

Proof: We represent the Rail-based interconnect as a directed capacitated graph $G = (V, E)$. Each node corresponds to either a GPU, a NIC, or a switch, and each directed edge $e \in E$ has transmission capacity c_e . Let $\mathbf{c} = (c_e)_{e \in E}$ and $\mathbf{f} = (f_e)_{e \in E}$ satisfy:

$$0 \leq f_e \leq c_e, \quad \forall e \in E \quad (8)$$

For inter-domain analysis, each domain d is modeled as a cluster of N GPUs with dedicated NICs ($\text{NIC}_{d,1}, \dots, \text{NIC}_{d,N}$). We contract the source domain k to a super-source s and the destination domain f to a super-sink t ; the validity of this contraction will be justified by a no-saturation argument under $R_1 > R_2$ below. Let $B \in \mathbb{R}^{|V| \times |E|}$ be the node-edge incidence matrix. Flow conservation is:

$$\mathbf{B}\mathbf{f} = \mathbf{b}, \quad \mathbf{b} = (\dots, \underbrace{+x}_{v=s}, \dots, \underbrace{-x}_{v=t}, \dots)^\top, \quad (9)$$

where x denotes the unidirectional throughput from k to f . The feasible region is:

$$\mathcal{F} = \{(x, \mathbf{f}) : \mathbf{B}\mathbf{f} = \mathbf{b}, \mathbf{0} \leq \mathbf{f} \leq \mathbf{c}\} \quad (10)$$

1) *Cut-Based Upper Bound:* For any s - t cut $(C, \bar{C})$ with $s \in C$, $t \in \bar{C}$, the forward cut-set is $\delta^+(C)$. By the Max-Flow-Min-Cut theorem, it has:

$$x \leq \text{Cap}(C) := \sum_{e \in \delta^+(C)} c_e \quad (11)$$

Choose C to include all nodes of domain k . The forward edges leaving C are exactly the N inter-domain uplinks:

$$E_{k \rightarrow \text{fabric}} = \{e_n : \text{NIC}_{k,n} \rightarrow S_n\}_{n=1}^N \quad (12)$$

where S_n denotes the leaf switch attached to $\text{NIC}_{k,n}$ (and symmetrically to $\text{NIC}_{f,n}$). Each e_n has capacity R_2 , so

$$\text{Cap}(C) = \sum_{n=1}^N c_{e_n} = N \cdot R_2 \quad (13)$$

Therefore:

$$\text{Cap}_{k \rightarrow f} \leq N \cdot R_2 \quad (14)$$

2) *Lower Bound:* Let $\mathcal{P}$ denote the set of simple $s \rightarrow t$ paths. We explicitly construct N rail-aligned paths:

$$\Pi_n : s \rightarrow \text{NIC}_{k,n} \rightarrow S_n \rightarrow \text{NIC}_{f,n} \rightarrow t, \quad n = 1, \dots, N \quad (15)$$

where each path Π_n uses a distinct NIC pair ($\text{NIC}_{k,n}, \text{NIC}_{f,n}$) and its associated leaf S_n . The NIC-leaf attachments are fixed per rail, so intra-domain edges used by different rails are edge-disjoint.

Assign rate $\alpha_n = R_2$ to each Π_n . Define the path-edge incidence variable $a_{e,n} \in \{0, 1\}$, which equals 1 if edge e lies on path Π_n and 0 otherwise. The induced edge load is:

$$f_e = \sum_{n=1}^N \alpha_n a_{e,n} = \sum_{n=1}^N R_2 a_{e,n} \quad (16)$$

By construction, all inter-domain edges are edge-disjoint across $\{\Pi_n\}_{n=1}^N$, and each such edge has capacity R_2 , hence $f_{e_n} = R_2 \leq c_{e_n} = R_2$.

Let E_{intra} denote the set of intra-domain edges. For intra-domain segments on Π_n , every traversed edge has capacity at least R_1 and carries only the rate of its own rail $\alpha_n = R_2$. Therefore, for each $n = 1, \dots, N$ and each intra-domain edge $e \in \Pi_n \cap E_{\text{intra}}$, we have:

$$f_e = R_2, \quad c_e \geq R_1 > R_2 \Rightarrow f_e < c_e \quad (17)$$

No intra-domain edge saturates under $R_1 > R_2$. Hence, contracting each domain to a super-node preserves the max-flow value. The aggregate throughput achieved by $\{\Pi_n\}$ is:

$$x = \sum_{n=1}^N \alpha_n = N \cdot R_2 \quad (18)$$

which proves:

$$Cap_{k \rightarrow f} \geq N \cdot R_2 \quad (19)$$

Combining (14) and (19) yields the equality.

3) *Engineering Note:* The theorem only assumes $R_1 > R_2$, whereas practical systems usually have $R_1 \gg R_2$ because intra-domain networks (e.g., NVLink) exceed NIC-based inter-domain links by a large margin, often close to an order of magnitude. Hence, inter-domain links form the bottleneck in practice, validating our parallel-rails view. ■

Summary. Rail-based network provides N independent logical rails, each with capacity R_2 , for any domain pair (k, f) . Therefore, the overall system capacity scales linearly with N , and subsequent optimization focuses on how to allocate traffic across these N rails to achieve global load balance and minimize all-to-all completion time.

C. Completion Time as a Load Balancing Problem

Building on Theorem 1, which establishes that inter-domain communication relies on N parallel rails, we now analyze the global all-to-all communication pattern. The objective is to minimize the overall completion time T^* . Since direct optimization of T^* is intractable, the problem is reformulated as an equivalent resource allocation problem in the form of load balancing across rails, which provides a more tractable analytical framework.

The all-to-all communication demand is described by the inter-domain traffic matrix $D^{(2)}$, while a specific flow allocation is characterized by the allocation matrix P . These jointly determine the per-domain sending and receiving loads $\mathbf{S}_{d,n}$ and $\mathbf{R}_{d,n}$ on each rail n . The system completion time is ultimately governed by the maximum processing time among all such loads. The following theorem formalizes this relationship between completion time and load distribution.

Theorem 2: Under the defined model and given the traffic matrix $D^{(2)}$, the minimum completion time T^* of the all-to-all communication is equivalent to solving a min-max load balancing problem over all NICs in both sending and receiving directions. If the rate of all inter-domain links is R_2 , then:

$$T^* = \frac{1}{R_2} \min_P \left\{ \max_{k,f,n} (\mathbf{S}_{k,n}, \mathbf{R}_{f,n}) \right\} \quad (20)$$

where $\min_P$ denotes the optimization over the strategy space formed by all feasible allocation matrices P , and $\max_{k,f,n}$ indicates the maximum taken over all sending and receiving loads.

Proof: This proof aims to demonstrate that any achievable completion time T is necessarily bounded below by the maximum load across all NICs, and that this lower bound can be attained by an optimal allocation.

For any given allocation strategy P , the corresponding load matrices $\mathbf{S}$ and $\mathbf{R}$ are fully determined. Completion of the all-to-all communication task implies that all sending and receiving operations on every track for every domain must be finished. Therefore, the total completion time T must be at least as large as the time required for any individual NIC to complete its assigned workload.

For any NIC(d, n), the time to complete its sending load $\mathbf{S}_{d,n}$ is $\mathbf{S}_{d,n}/R_2$, and the time to complete its receiving load $\mathbf{R}_{d,n}$ is $\mathbf{R}_{d,n}/R_2$. Hence, we have:

$$T \geq \frac{\mathbf{S}_{d,n}}{R_2} \quad \text{and} \quad T \geq \frac{\mathbf{R}_{d,n}}{R_2} \quad (21)$$

Considering all NICs, the completion time under this strategy must satisfy:

$$T \geq \frac{1}{R_2} \max_{k,f,n} (\mathbf{S}_{k,n}, \mathbf{R}_{f,n}) \quad (22)$$

This inequality holds for all feasible allocation strategies P . Therefore, the global minimum completion time T^* cannot be smaller than the optimal value obtained by minimizing this maximum load over all strategies:

$$T^* \geq \frac{1}{R_2} \min_P \left\{ \max_{k,f,n} (\mathbf{S}_{k,n}, \mathbf{R}_{f,n}) \right\} \quad (23)$$

The above derivation establishes a lower bound. When traffic can be continuously divided, the search for an optimal allocation matrix P minimizing the load $\max(\mathbf{S}, \mathbf{R})$ reduces to a linear programming problem. Standard linear programming theory ensures the existence of an optimal allocation P^* whose induced maximum load attains the min-max optimum. Under this allocation, the system completion time reaches the lower bound, and the inequality becomes an equality.

In summary, the problem of minimizing the all-to-all time is rigorously proven to be equivalent to minimizing the maximum sending and receiving load across all NICs. ■

Summary. This theorem transforms the dynamic performance metric into a optimizable load objective, guiding the search for an optimal traffic allocation strategy P that minimizes the maximum NIC load. To solve this min-max problem systematically, it must first be formulated as a mathematical program. The next section presents the linear programming formulation that underlies the optimal allocation strategy.

D. Objectives and Constraints

Based on the conclusions of the previous section, we formulate the min-max problem of all-to-all minimum completion time as a standard linear programming (LP) model to facilitate formal solution. The model takes the allocation matrix P as the core decision variable and introduces an auxiliary variable t representing the upper bound of all loads.

The optimization problem is formulated as:

$$\begin{aligned} \min_{P,t} \quad & t \\ \text{s.t.} \quad & \sum_{f=1}^M D_{k,f}^{(2)} \cdot P_{k,f,n} \leq t, \quad \forall k, n \\ & \sum_{k=1}^M D_{k,f}^{(2)} \cdot P_{k,f,n} \leq t, \quad \forall f, n \\ & \sum_{n=1}^N P_{k,f,n} = 1, \quad \forall k, f \\ & P_{k,f,n} \geq 0, \quad \forall k, f, n \end{aligned} \quad (24)$$

In this LP formulation, the auxiliary variable t represents the objective to be minimized and is defined as the global upper bound of all NIC sending loads $\mathbf{S}_{k,n}$ and receiving loads $\mathbf{R}_{f,n}$. Minimizing t therefore corresponds to minimizing the system's bottleneck load. The remaining constraints guarantee a valid allocation matrix P , ensuring complete traffic assignment (normalization) and correct directionality (non-negativity). As both the objective and constraints are linear, the problem can be solved optimally using standard LP methods.

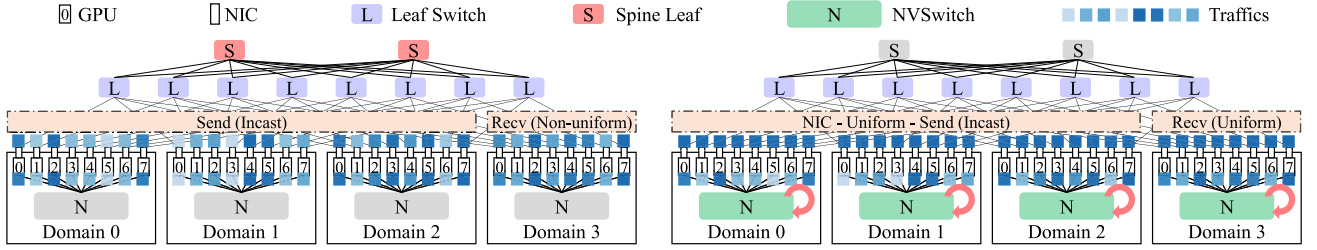


Fig. 8. Illustration of symmetry mechanism. The left figure does not leverage the topology. The right figure leverages the intra-domain forwarding and topological properties, where uniform NIC sending results in uniform receiving.

This LP formulation provides a numerical solution to the problem, while the next section will further explore and present a more insightful analytical solution.

E. Topological Symmetry of Load

The all-to-all minimum completion time problem has been formulated as a linear program. Here we derive a closed-form analytical solution and prove its optimality. The key insight is that in the Rail-based architecture, uniform sending loads inherently yield uniform receiving loads in Fig. 8.

Theorem 3: In the studied system, any allocation strategy that achieves uniformization of sending loads is globally optimal. Specifically, if for every source domain k , the sending load is uniformly distributed as:

$$S_{k,n} = \frac{1}{N} \sum_{f=1}^M D_{k,f}^{(2)}, \quad \forall n = 1, \dots, N \quad (25)$$

then the receiving load for any destination domain f is automatically uniformized:

$$R_{f,n} = \frac{1}{N} \sum_{k=1}^M D_{k,f}^{(2)}, \quad \forall n = 1, \dots, N \quad (26)$$

This strategy simultaneously minimizes both sending and receiving bottlenecks, thereby achieving the all-to-all minimum completion time T^* .

Proof: The core of this proof relies on the system structure revealed in Theorem 1. Communication from any source domain k to destination domain f is equivalent to N parallel, independent paths, with endpoints $NIC_{k,n}$ and $NIC_{f,n}$ for $n = 1, \dots, N$. This implies that traffic sent from $NIC_{k,n}$ along the n -th track is uniquely directed to $NIC_{f,n}$.

We first construct a sending-uniform strategy. This strategy evenly distributes the total outgoing traffic of each source domain k , $\sum_{f=1}^M D_{k,f}^{(2)}$, across its N NICs. Specifically, the sending load on the n -th NIC is set equal to the average load. By adopting the allocation strategy:

$$P_{k,f,n}^* = \frac{1}{N} \quad (27)$$

we obtain the following equation:

$$S_{k,n} = \sum_{f=1}^M D_{k,f}^{(2)} P_{k,f,n}^* = \frac{1}{N} \sum_{f=1}^M D_{k,f}^{(2)} \quad (28)$$

This strategy successfully uniformizes the sending load, thereby minimizing the maximum processing time among all sending tasks.

Next, we examine the impact of this strategy on receiving loads. The receiving load $R_{f,n}$ on the n -th track for any destination domain f is the sum of the traffic from all source domains k through that track. Based on the one-to-one correspondence of paths in the Rail architecture, it has:

$$R_{f,n} = \sum_{k=1}^M D_{k,f}^{(2)} P_{k,f,n}^* = \frac{1}{N} \sum_{k=1}^M D_{k,f}^{(2)} \quad (29)$$

This shows that the receiving load is automatically uniformized, with each receiving NIC carrying exactly the average of the domain's total incoming traffic.

Thus, a simple sending-uniform strategy, due to the intrinsic system architecture, inevitably leads to uniform receiving loads. Uniform sending minimizes the sending bottleneck, while the resulting uniform receiving minimizes the receiving bottleneck. Since this single strategy simultaneously optimizes both sending and receiving loads, it is globally optimal and achieves the all-to-all minimum completion time T^* .

Consequently, the optimal allocation matrix achieving this state has a simple closed-form solution:

$$P_{k,f,n}^* = \frac{1}{N}, \quad \forall k, f, n \quad (30)$$

which satisfies the normalization constraint:

$$\sum_{n=1}^N P_{k,f,n}^* = 1 \quad (31)$$

The proof is complete. $\blacksquare$

Summary. This theorem provides the optimal allocation strategy under the ideal assumption that traffic can be arbitrarily split. However, in practical systems, traffic often exists as discrete units that cannot be fully subdivided. The next section investigates how to effectively approximate this ideal uniform allocation strategy under such discrete constraints.

F. LPT-Based Load Balancing for Atomic Flows

The previous section derived the optimal allocation under the ideal assumption of arbitrarily divisible traffic. In practice, however, communication between GPUs is scheduled as indivisible units, making continuous allocation infeasible. Even when flows are split, they remain discrete, as large flows are only divided into several smaller ones (packets or flowlets) rather than continuous segments (bytes or bits).

This section addresses the load balancing problem under such discrete constraints. We aim to minimize the mean squared error between the actual loads and the ideal uniform loads guided by Theorem 3. To this end, we propose an efficient approximate allocation algorithm and provide theoretical guarantees for its performance.

We assume that each flow is indivisible or has been split into the smallest atomic units. The inter-domain total traffic matrix $D^{(2)}$ is known, i.e., the flow between each source domain k and destination domain f , $D_{k,f}^{(2)}$, is given. Based on a fixed allocation matrix P , let p_n denote the fraction of traffic ideally assigned to the n -th NIC. Then the target load for each flow on NIC n is defined as:

$$T_{k,n} = \sum_{f=1}^M D_{k,f}^{(2)} p_n \quad (32)$$

Each GPU-to-GPU flow must be assigned entirely to a single NIC. The problem can thus be formalized as allocating a set of discrete flows with weights w_i to N NICs, such that the actual load on each NIC

$$L_j = \sum_i w_i x_{i,j}, \quad x_{i,j} \in \{0, 1\} \quad (33)$$

is as close as possible to its target load T_j . The binary assignment variables $x_{i,j}$ satisfy:

$$x_{i,j} = \begin{cases} 1, & \text{flow } i \text{ on NIC } j \\ 0, & \text{otherwise} \end{cases}, \quad \sum_{j=1}^N x_{i,j} = 1, \quad \forall i \quad (34)$$

To quantify the load deviation, we introduce the MSE as the metric:

$$\text{MSE} = \frac{1}{N} \sum_{j=1}^N (L_j - T_j)^2 \quad (35)$$

Under this definition, the GPU-to-GPU flow allocation problem with fixed allocation fractions p_n can be formalized as a combinatorial optimization problem:

$$\begin{aligned} \min_{x_{i,j}} \quad & \frac{1}{N} \sum_{j=1}^N (L_j - T_j)^2 \\ \text{s.t.} \quad & \sum_{j=1}^N x_{i,j} = 1, \quad \forall i, \\ & L_j = \sum_i w_i x_{i,j}, \quad \forall j, \\ & x_{i,j} \in \{0, 1\}, \quad \forall i, j \end{aligned} \quad (36)$$

This problem is a discrete combinatorial optimization task, aiming to minimize the mean squared deviation between the actual load and the target load on each NIC. Since each flow is assigned to exactly one NIC, directly solving this problem incurs high computational complexity in large-scale systems. To reduce computational overhead, we adopt the LPT for approximate allocation.

The LPT method first sorts all flows in descending order of weights w_i and then sequentially assigns each flow to the NIC with the currently smallest total load until all flows are allocated. According to Theorem 3, the sending-uniform strategy ensures that the target load T_j already matches the ideal average load $L_{\text{opt}} = \sum_i w_i / N$, so the LPT algorithm effectively performs discrete allocation while maximizing the utilization of uniformly balanced NIC loads.

Theorem 4: For the discrete-flow load balancing problem described above, when the LPT algorithm is employed for allocation, the MSE between the actual load and the ideal target load is upper-bounded by the square of the largest single flow weight $w_{\max}$:

$$\text{MSE} \leq w_{\max}^2 \quad (37)$$

Proof: Classical LPT theory [16] guarantees that, for the standard discrete load balancing problem, the maximum load $L_{\max}$ satisfies:

$$L_{\max} \leq L_{\text{opt}} + w_{\max} \quad (38)$$

where $w_{\max} = \max_i w_i$ denotes the weight of the largest single flow. The LPT approximation ratio is given by:

$$R = \frac{L_{\max}}{L_{\text{opt}}} \leq \frac{4}{3} - \frac{1}{3N} \quad (39)$$

Consequently, the deviation of the actual load on each NIC from the target load is bounded by:

$$|L_j - L_{\text{opt}}| \leq L_{\max} - L_{\text{opt}} \leq w_{\max}, \quad \forall j. \quad (40)$$

This directly leads to the following upper bound on MSE:

$$\text{MSE} = \frac{1}{N} \sum_{j=1}^N (L_j - L_{\text{opt}})^2 \leq w_{\max}^2 \quad (41)$$

Thus, by sorting flows in descending order and greedily assigning them to the NIC with the current minimum load, LPT effectively minimizes the load deviation of each NIC and indirectly controls the MSE. ■

Complexity Analysis. The computational complexity of the RailS algorithm is primarily dictated by two steps: sorting and assignment. First, sorting the set of F atomic flows by their weights requires $O(F \log F)$ time. Second, assigning each flow to one of the N rails involves finding the rail with the minimum accumulated load. A linear scan over N rails for each of the F flows results in $O(FN)$ operations. Consequently, the overall time complexity is $O(F \log F + FN)$.

In practical MoE training clusters, the number of rails N is determined by the hardware topology (e.g., $N = 8$ for typical GPU nodes), which is a small constant. Thus, the scheduling overhead is computationally lightweight and efficient enough to be executed on the host CPU in real-time, causing negligible delay compared to the actual data transmission time. This contrasts sharply with the exponential complexity of exact ILP solutions.

Summary. This section extends the continuous-flow theory to practical systems with indivisible flows. Through the LPT algorithm, we show that discrete allocation can still achieve predictable load balancing. The key insight is that load balancing effectiveness depends on the maximum flow granularity, motivating the next focus on designing flow-splitting and scheduling mechanisms that control granularity to approach the theoretical optimum.

V. DESIGN AND IMPLEMENTATION

Building upon the theoretical framework established in the previous section, this section presents the concrete design and implementation of RailS. Each system module is directly informed by the theoretical proofs in Section IV to ensure that the implementation approaches the optimal bounds. Specifically, Multipath Transmission operationalizes Theorem 1 by activating all physical rails to realize the existence of N parallel logical channels; Proactive Spraying leverages Theorem 3 (Symmetry), which guarantees that uniform sending leads to uniform receiving, allowing the system to rely on local proactive scheduling rather than complex global coordination; and Flow Splitting and LPT implements Theorem 4, where the load imbalance is bounded by $w_{\max}^2$, motivating the use of flow splitting to granularity ($w_{\max}$) and the LPT algorithm to minimize the time.

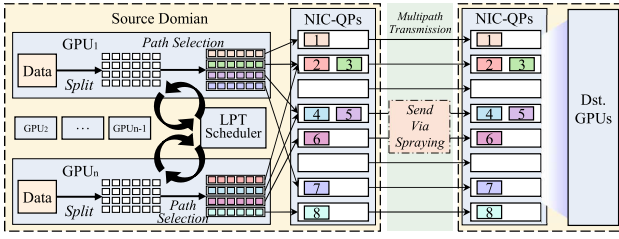


Fig. 9. System design and implementation.

A. Multi-Path Spraying Framework

To fully exploit the N parallel communication lanes provided by the Rail architecture, our system employs multi-path packet spraying as the core traffic distribution mechanism in Fig. 9. Spraying is an advanced multi-path transmission technique designed to actively disperse the units of a logical data flow (e.g., packets or subflows) across multiple physical paths. For the sparse and uneven all-to-all traffic generated by MoE models, the spraying mechanism provides finer-grained and more dynamic load balancing compared to traditional ECMP static hashing, effectively alleviating link congestion caused by hash collisions on large flows.

Our system builds on a scalable software transport framework that supports flexible, multi-path data transmission across NICs. Algorithm 1 presents the procedure for system operation in MoE training. Its primary advantage lies in its successful decoupling of the RDMA NIC's control and data paths, enabling flexible implementation and innovation of transport-layer protocols in software on the host CPU. This design provides a natural entry point for integrating a custom LPT scheduler. The framework mainly exploits some key features

Multipath Transmission. The system realizes multipath communication by establishing and managing multiple QPs between each pair of communicating NICs. In our Rail architecture, each node is equipped with N NICs, and this mechanism is used to construct N parallel logical communication lanes, perfectly corresponding to the N parallel channels in our theoretical model. By default, it supports up to 256 QPs for parallelization and load balancing, providing abundant path resources for traffic spraying.

Flow Splitting and Atomicity. Theorem 4 provides a strict theoretical bound on load imbalance, proving that the MSE is upper-bounded by the square of the maximum flow size (w_{max}^2). This insight dictates our engineering requirement for flow atomicity: to minimize imbalance, we must actively reduce w_{max} . Consequently, the coalescing mechanism segments large messages into fixed-size atomic flows (e.g., 32 KB) to ensure the system strictly adheres to the performance bounds predicted by the theorem. In our system, these generated fixed-size chunks, or small application-layer messages, are treated as indivisible atomic flows. These atomic flows serve as the fundamental units for load balancing operations.

Proactive Spraying. Current spraying strategies are often reactive, such as randomly sampling path RTTs and probabilistically steering traffic to currently lower-latency paths. While these approaches can adapt to network dynamics, they do not necessarily achieve the globally balanced load. Guided by Theorem 3, RailS adopts a proactive strategy. Since it proves that uniform sending inherently leads to uniform receiving in Rail architectures, the complex problem of global load balancing is mathematically reduced to optimizing local sending distributions. This theoretical guarantee allows our system to

Algorithm 1 MoE Distributed Training With DP/EP/PP and LPT-Based All-to-All Communication

Input: Local GPU g ; Number of rails N ; model parameters Θ ; Expert assignment $\mathcal{E}$

Output: Updated model parameters Θ after one training iteration

- ```

//Step 0: Preprocessing / Input preparation
1 Load mini-batch input data for this GPU.
//Step 1: Attention and gating computation
2 Compute attention outputs: $AttnOut \leftarrow Attention(Inputs, \Theta)$
3 Compute gating decisions: $Gate \leftarrow Gate(AttnOut)$
//Step 2: First All-to-All (inputs to experts)
4 Select input slices for local experts according to $Gate$ and $\mathcal{E}$
5 Split inputs into atomic flows $\{w_i\}$
6 Call LPT Scheduler to assign flows to rails
7 Transmit flows via spraying
//Step 3: Expert computation (distributed EP)
8 Compute local expert outputs $ExpOut_e$
9 If expert spans multiple GPUs, aggregate partial results
//Step 4: Second All-to-All (expert outputs aggregation)
10 Split expert outputs into atomic flows $\{w_i\}$
11 Call LPT Scheduler to assign flows to rails
12 Transmit flows via spraying
//Step 5: Add & Norm computation
13 $Output \leftarrow AddNorm(AttnOut, ExpOut)$
//Step 6: Output propagation
14 $Out_{MoE} \leftarrow Output$
15 Pass Out_{MoE} to the next pipeline stage

```

utilize a deterministic scheduler that relies solely on local information, avoiding the latency and overhead associated with reactive feedback loops. Using a deterministic scheduling algorithm, the system directly computes the optimal target path for each atomic flow. The next section presents the key component that realizes the LPT scheduler.

To integrate these components into a coherent workflow, Algorithm 1 details the execution sequence of a RailS-enabled MoE training iteration.

Initially, the system performs standard attention and gating computations to determine token routing (Lines 1–3). Once the routing matrix is established, the critical Dispatch phase begins. Instead of immediately sending bulk data, the Flow Splitting mechanism decomposes the large messages into atomic flows based on the granularity defined by Theorem 4 (Line 4–5).

Subsequently, the LPT Scheduler is invoked (Line 6) to calculate the optimal path assignment for these atomic flows using the sorting and greedy assignment logic. With the path assignments fixed, the multipath transmission module proactively sprays the flows across the  $N$  parallel rails (Line 7).

After the experts complete their computations (Lines 8–9), the Combine phase mirrors this process: expert outputs are

again split into atomic flows (Line 10), scheduled via LPT (Line 11), and sprayed back to the source domains (Line 12), ensuring load balance is maintained in both communication directions.

### B. LPT Scheduler

The scheduler serves as the core decision engine, translating the theoretical algorithm from section IV into engineering practice. Recall from Theorem 2 that minimizing completion time is equivalent to a min-max load balancing problem. While exact solutions are computationally expensive, Theorem 4 validates that the LPT algorithm provides a tight approximation ratio. Therefore, the scheduler implements the specific sorting and greedy assignment logic of LPT to minimize the maximum load on any single Rail, ensuring the actual completion time approaches the theoretical lower bound.

**Architecture and State Management.** The LPT Scheduler runs independently on each sending node. Its core component is a local load state array of size  $N$ , denoted as  $LoadState[N]$ . Each entry  $LoadState[j]$  maintains the cumulative number of bytes already assigned to the  $j$ -th communication lane (i.e., the  $j$ -th local NIC) for the current all-to-all operation. To adapt to the dynamic nature of MoE traffic, the array is reset at the start of each new all-to-all round, ensuring that scheduling decisions are always based on the current communication round's status.

**Workflow and Implementation.** Upon the initiation of an all-to-all communication by the upper-layer application, the generated traffic is split into a set of atomic flows  $\{w_i\}$ . The LPT Scheduler then takes over the path selection for these flows, following the LPT algorithm 2 strictly:

- **Flow Sorting:** The scheduler first sorts all atomic flows  $\{w_i\}$  to be sent from the local node in descending order of their weights.
- **Iterative Assignment:** The scheduler then iterates over the sorted flow list. For each flow  $w_i$ , it performs the following core steps:
  - *Path Selection:* Identify the index of the path with the currently minimum cumulative load in the local  $LoadState$  array  $j^* = \arg \min_j LoadState[j]$ .
  - *Decision Output:* Assign  $w_i$  to the selected path  $j^*$  and return this decision to transport engine.
  - *State Update:* Immediately update the local load state  $LoadState[j^*] \leftarrow LoadState[j^*] + w_i$ .
- **Spraying Execution:** Upon receiving the path index  $j^*$ , the transport engine transmits the atomic flow  $w_i$  through the set of QPs associated with the  $j^*$ -th lane. Repeating this process for all atomic flows completes a global, deterministic spraying, systematically distributing the node's total outbound traffic across the  $N$  parallel lanes according to the LPT-optimized allocation.

The LPT Scheduler effectively translates the theoretical model of Theorem 4 into a fully distributed low-complexity engineering implementation. Unlike naive random spraying, it performs strategic flow assignments aimed at minimizing load variance, thereby fully exploiting the multi-NIC parallelism provided by the Rail architecture, even under the irregular and sparse traffic patterns typical of MoE workloads.

## VI. EVALUATION

### A. Experimental Setup

**Simulation Setup.** We implement a scalable datacenter simulation environment on Mininet 2.3.1b4 with Soft-RoCE,

### Algorithm 2 LPT Scheduler

---

**Input:** Number of rails  $N$ ; set of local GPUs  $\mathcal{G}$   
**Output:** Path assignment  $\{(w_i, j^*)\}$  for all flows; load MSE  
 // Initialization before each All-to-All

- 1 Initialize  $LoadState[1 \dots N] \leftarrow 0$ .
- 2 Initialize empty flow set  $\mathcal{W} \leftarrow \emptyset$ .
- // Step 1: Flow collection from local GPUs
- 3 **for each GPU**  $g \in \mathcal{G}$  **do**
- 4   Receive atomic flows  $\{w_{g,1}, w_{g,2}, \dots\}$  from GPU  $g$ .
- 5   Tag each  $w_{g,k}$  with source ID  $g$ .
- 6   Append them into  $\mathcal{W}$ .
- // Step 2: LPT sorting
- 7 Sort  $\mathcal{W}$  by descending weight.
- 8 Break ties by GPU index.
- // Step 3: Iterative allocation
- 9 **for each flow**  $w_i \in \mathcal{W}$  **do**
- 10    $j^* \leftarrow \arg \min_{j \in [1, N]} LoadState[j]$ .
- 11   Record mapping  $(w_i, j^*)$  into assignment table.
- 12   Notify source GPU of  $w_i$  with selected NIC  $j^*$ .
- 13    $LoadState[j^*] \leftarrow LoadState[j^*] + w_i$ .
- // Step 4: Spraying execution
- 14 **for each assignment**  $(w_i, j^*)$  **do**
- 15   Select port  $p$  from NIC  $j^*$  by round-robin.
- 16   Map  $(j^*, p)$  to a QP in NIC's QP set.
- 17   Bind  $w_i$  to the chosen QP.
- 18   Transmit  $w_i$  via transport engine.
- // Step 5: Completion & Feedback
- 19 **for each flow**  $w_i$  **do**
- 20   Await completion signal from NIC  $j^*$ .
- 21   Confirm delivery to destination GPU.
- 22   Update scheduler statistics with actual transmission time and throughput.
- // Step 6: Performance evaluation (MSE)
- 23 Compute average load:  $\mu \leftarrow \frac{1}{N} \sum_{j=1}^N LoadState[j]$
- 24 Compute  $MSE \leftarrow \frac{1}{N} \sum_{j=1}^N (LoadState[j] - \mu)^2$
- 25 Log  $MSE$  for analysis and scheduler tuning.

---

using a Rail-optimized topology. The network comprises 16 spine switches and 32 fully connected leaf switches. Each computing domain connects in parallel to different leaf switches via multiple NICs, forming Rail-based links. The system scales to 128 compute domains with eight GPUs each interconnected through NVLINK. To accommodate this scale, each of the 32 leaf switches provides 32 downlink ports. We configure all link bandwidths to 100 Gbps, and the external link propagation delay is set to 5  $\mu$ s, while intra-server delay is assumed to be negligible. For congestion emulation, we implement a Fair Queuing model with First-In-First-Out processing, utilizing a switch buffer size of 500 KB per queue and ECN marking thresholds ( $K_{\min} = 30$  KB,  $K_{\max} = 65$  KB). Furthermore, a 2:1 oversubscription ratio is enforced at the leaf tier to intentionally simulate realistic network congestion and micro-burst scenarios. Network forwarding uses Open vSwitch with OpenFlow for fine-grained control, while a Ryu-based distributed controller installs flow rules and manages ECMP.

**Workloads.** We construct four representative workloads using MoE outputs from the SimAI AICB library [61] with controlled token and gating distributions to evaluate RailS across a spectrum of traffic patterns. Table I summarizes the corresponding distributions.

- **Uniform Workload.** This scenario involves every sender transmitting an equal volume of data to all receivers and yields a perfectly balanced link load. This pattern



TABLE I  
THE MOE WORKLOADS' TOKEN AND GATING DISTRIBUTIONS

| Type<br>Distribution | Uniform | Sparse  | Sender-<br>skewed | Receiver-<br>skewed | Real<br>workload   |
|----------------------|---------|---------|-------------------|---------------------|--------------------|
| <b>Token input</b>   | Uniform | Uniform | Zipf              | Uniform             | Uniform            |
| <b>Gating</b>        | Uniform | Top-K   | Uniform           | Zipf                | Training-<br>based |

physically represents the communication characteristics of traditional dense models and hybrid large models that incorporate both dense and sparse layers. It further simulates an ideal MoE model where an auxiliary loss function successfully enforces a perfectly uniform expert selection. We utilize this workload as a baseline to determine the maximum achievable bus bandwidth and to verify system overhead in the absence of traffic skew.

- **Sparse Workload.** This workload applies a Top-k expert selection matrix with column wise sparsity to simulate naturally uneven communication patterns. It mimics standard routing where specific domain experts become popular and handle proportionally more traffic. This setting represents a state where multiple distributed hotspots exist but the network remains functional rather than overwhelmed. We use this scenario to evaluate the ability of the system to handle dynamic routing logic under realistic traffic imbalances.
- **Skewed Workload.** This model employs a Zipfian distribution to simulate extreme imbalance and worst case scenarios in two distinct forms. The receiver skew models an incast scenario where a domain specific batch such as a sequence consisting entirely of Python code causes the gating network to direct all tokens to a single expert. The sender skew simulates uneven input distribution resulting from insufficient data shuffling. This workload differs from the sparse setting by creating single point breakdowns and severe congestion to strictly test the robustness of congestion control algorithms and tail latency performance.
- **Real MoE Workload.** We replay traffic traces from the training of Mixtral  $8 \times 7B$  using MixNet [38] to reproduce realistic communication patterns. Unlike synthetic workloads this trace captures complex upper layer strategies such as capacity factors and greedy policies along with temporal correlations between iterations. It provides a comprehensive validation of RailS in a production environment where temporal dynamics and engineering constraints play a critical role.

**Evaluation Metrics.** We evaluate performance using some key metrics. **Collective Completion Time (CCT)** measures MoE all-to-all efficiency across average, 80th, 95th, and 99th percentiles, with the 99th percentile approximating total transfer completion. **Bus Bandwidth (BusBw)** quantifies effective link utilization and enables direct comparison with peak hardware bandwidth to assess algorithmic efficiency. **NIC Transmission/Reception Volume** captures per-NIC send and receive traffic via a matrix representation, assessing load balance across interfaces. **Normalized MSE of Load** evaluates intra-domain NIC load balance on a 0–1 scale, where 0 denotes perfect uniformity. **Iteration Time** measures the total duration of one training iteration.

**Baseline Methods.** We propose the **RailS** and compare it with four baseline methods: **ECMP** [24] is a widely used

equal-cost multipath routing mechanism in datacenters that binds flows to a single path via hashing, providing traditional load balancing and serving as a performance benchmark. **MinRTT** [45] is a multipath scheduling algorithm based on RTT measurements. It selects the subflow with the minimum RTT to transmit packets, enabling multipath bandwidth aggregation and load balancing. MinRTT was first implemented in MPTCP. **PLB** [48] is a flowlet-based load balancing scheme that uses IPv6 flow label to dynamically switch paths during idle periods, balancing loads while minimizing packet reordering. **REPS** [8] is a per-packet spraying load balancing algorithm that minimally manages state to reroute packets around congestion hotspots and unreliable links.

### B. Performance Under Uniform Load

The uniform-load scenario simulates an ideal condition in which each sender transmits an equal amount of data to all receivers, resulting in balanced link utilization without the presence of “elephant” or “mice” flows.

**BusBw.** Fig. 10a shows the normalized BusBw of different schemes relative to ECMP under the uniform-load scenario. RailS achieves a normalized BusBw of 1.29, representing an improvement of approximately 29% over ECMP, demonstrating its superior bandwidth utilization. REPS and MinRTT achieve normalized bandwidths of 1.34 and 1.24, respectively, which are close to that of RailS. PLB’s normalized BusBw is 20% lower than that of RailS, indicating relatively average performance.

**CDF of CCT.** Fig. 11a illustrates the CDF of normalized CCT for different schemes relative to RailS under the uniform-load scenario. Specifically, RailS and REPS exhibit similar and concentrated trends, reflecting excellent load-balancing capability. MinRTT completes some flows faster, but its overall completion time is slightly worse than RailS. PLB follows, showing better performance than ECMP.

**CCT Values.** Fig. 12a further presents the mean, 80th, 95th, and 99th percentile values of normalized CCT for different schemes under uniform load. RailS demonstrates very close mean and tail percentiles, indicating good load-balancing capability. REPS performs consistently with RailS. Although MinRTT achieves the lowest mean CCT, its overall completion time is 4% worse than RailS. The tail flows of PLB finish relatively simultaneously, with an overall completion time 18% longer than RailS but still 11% better than the worst-performing ECMP.

### C. Performance Under Sparse Load

The sparse-load scenario is constructed using a Top-K ( $K = 2$ ) expert selection matrix to simulate non-uniform communication patterns. By adjusting the sparsity, only a subset of receivers participates in data processing, forming hotspot experts and unbalanced link loads. This setup is used to evaluate the load-balancing capability of different schemes under non-ideal conditions.

**BusBw.** Fig. 10b, 10c, 10d and 10e show normalized BusBw of different schemes under various sparsity levels (0.6, 0.4, 0.2, and fully dense). RailS consistently maintains the highest BusBw across all sparsity levels, and its advantage becomes more pronounced as the load becomes sparser. At a sparsity of 0.6, RailS achieves a BusBw  $2.8\times$  higher than other schemes. Under other sparsity levels, RailS still achieves 20%–78% performance improvements, indicating that RailS fully utilizes



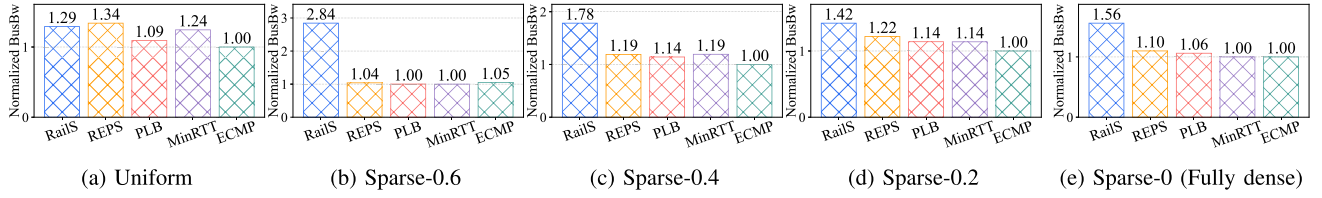


Fig. 10. Normalized BusBw of different schemes under various all-to-all workloads: (a) Uniform matrix, (b) Sparse-0.6, (c) Sparse-0.4, (d) Sparse-0.2, and (e) Sparse-0 (fully dense).

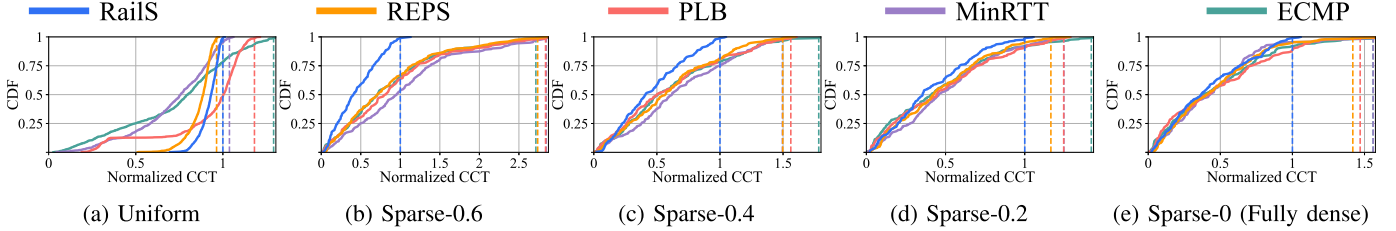


Fig. 11. CDF of normalized CCT for different schemes under various all-to-all workloads: (a) Uniform matrix, (b) Sparse-0.6, (c) Sparse-0.4, (d) Sparse-0.2, and (e) Sparse-0 (fully dense).

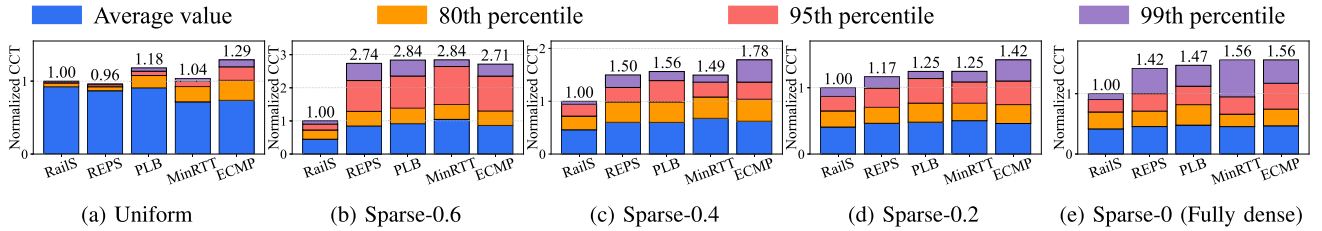


Fig. 12. Normalized CCT of different schemes under various all-to-all workloads, showing average, 80th, 95th, and 99th percentiles: (a) Uniform matrix, (b) Sparse-0.6, (c) Sparse-0.4, (d) Sparse-0.2, and (e) Sparse-0 (fully dense).

the multi-path bandwidth of the Rail architecture to maintain high BusBw.

**CDF of CCT.** Fig. 11b, 11c, 11d and 11e illustrate the CDF of normalized CCT for different schemes under sparse load. It is evident that as sparsity decreases, RailS completes flows faster and with greater advantage. With increasing sparsity, the short-flow performance of RailS becomes closer to other schemes, yet the longest flows still complete the fastest. This demonstrates that RailS achieves optimal load balancing and maintains excellent completion time performance.

**CCT Values.** Fig. 12b, 12c, 12d and 12e further present the mean, 80th, 95th, and 99th percentile values of normalized CCT for different schemes under sparse load. At low sparsity, RailS maintains closely aligned tail flow completion times. As sparsity increases, the range of tail completion times expands. Compared with other schemes, RailS consistently achieves the best mean and percentile performance in normalized CCT. At a sparsity of 0.6, RailS improves total completion time by more than 1.5 $\times$ . With higher sparsity levels, the improvement remains within 17%–78%, demonstrating the superior load-balancing performance of RailS.

#### D. Performance Under Skewed Sender Load

The sender-skewed scenario employs a Zipf distribution to make a small number of domain experts carry the majority of traffic, forming “hotspot senders.” This simulates a situation where non-uniform input leads to extremely unbalanced outgoing loads among experts. It is used to examine each scheme’s bandwidth utilization and congestion mitigation capability under sudden source-side pressure.

**BusBw.** Fig. 13a shows that RailS and REPS achieve BusBw 2.7 $\times$  and 2.75 $\times$  higher than PLB, respectively. RailS improves bandwidth by 126% compared with ECMP and by 58% compared with MinRTT, demonstrating the effectiveness of the spraying mechanism under sender-skewed load.

**Sender Load Matrix and MSE.** The heatmap in Fig. 13b shows that RailS and REPS maintain balanced traffic distribution across NICs within each domain, with an inter-domain MSE below 0.01, indicating excellent sender load balancing. MinRTT shows moderate balance with an MSE of about 0.03, while PLB and ECMP have the highest MSE, around 15%.

**Receiver Load Matrix and MSE.** Fig. 13c shows that although the sending side is skewed, the receiving traffic distribution remains uniform. As a result, the actual NIC-level receiving load of all schemes is balanced, and the bottleneck lies on the sender side.

**CCT.** As shown in Fig. 13d, the CDF curves indicate that RailS and REPS achieve the best CCT performance, reducing completion time by 55.9%–62.9% compared with ECMP and PLB, and by 34.6% compared with MinRTT.

In summary, RailS and REPS effectively mitigate sender-side bottlenecks under skewed load through the spraying mechanism, achieving balanced transmission. MinRTT benefits from multipath utilization and thus outperforms PLB and ECMP. However, since the bottleneck is not within the network and PLB cannot exploit the Rail architecture’s advantages, its performance is the lowest.

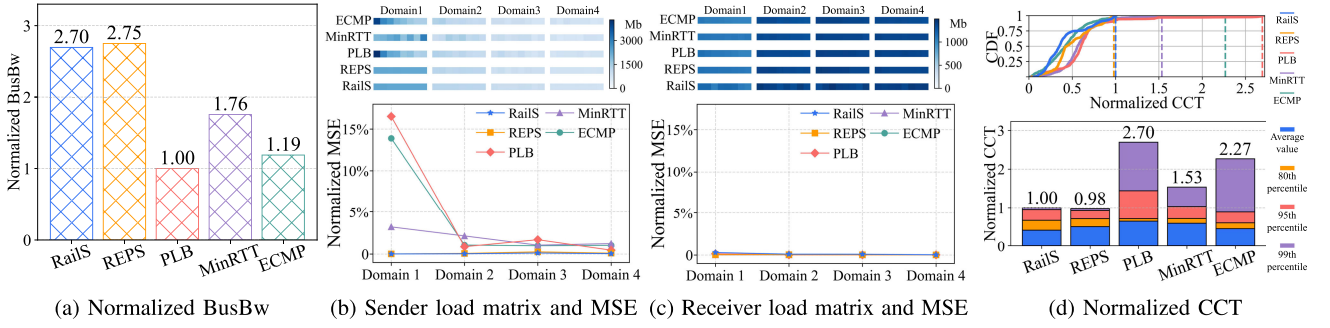


Fig. 13. Performance of different schemes under sender-skewed workloads: (a) normalized BusBw, (b) sender load matrix and its MSE, (c) receiver load matrix and its MSE, and (d) normalized CCT.

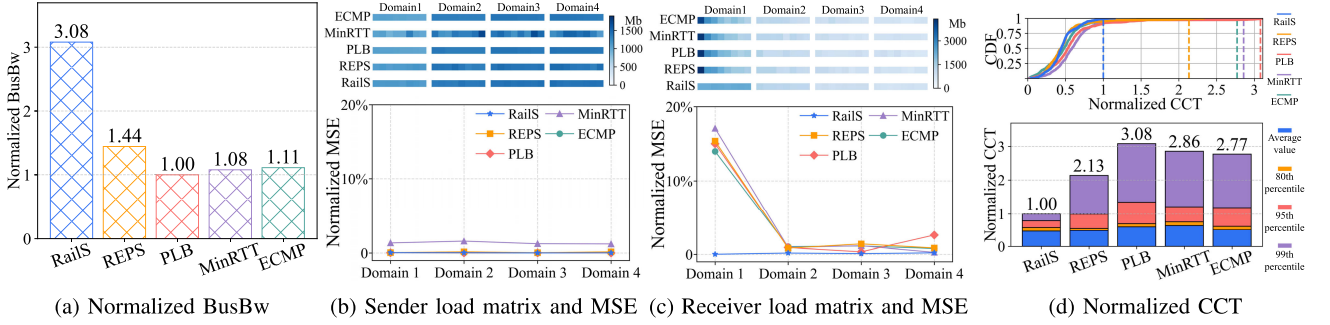


Fig. 14. Performance of different schemes under receiver-skewed workloads: (a) normalized BusBw, (b) sender load matrix and its MSE, (c) receiver load matrix and its MSE, and (d) normalized CCT.

### E. Performance Under Skewed Receiver Load

The receiver-skewed scenario models an *incast* situation, where most senders concentrate on a few hot experts following a Zipf distribution, leading to sudden many-to-one traffic bursts. This pattern reflects the real training condition in which “hot experts” are frequently invoked. It is used to evaluate each scheme’s ability to suppress tail latency and maintain load balance when the target-side load increases sharply.

**BusBw.** As shown in Fig. 14a, RailS achieves  $3.08\times$  normalized BusBw, representing a  $2\times$  improvement over ECMP, MinRTT, and PLB, significantly alleviating receiver hotspots. REPS reaches only  $1.44\times$  that of PLB, less than half of RailS.

**Sender Load Matrix and MSE.** Fig. 14b shows that although the receiver load is skewed, the sending traffic distribution remains uniform. Therefore, all schemes exhibit balanced NIC-level sending and receiving loads, with the bottleneck located on the receiver side.

**Receiver Load Matrix and MSE.** Fig. 14c demonstrates that RailS maintains balanced receiving traffic across domains, with an MSE below 0.01, while all other schemes have MSE values around 15%. The heatmap clearly shows that although Domain 1 experiences a strong receiving skew, RailS effectively evens it out. This indicates that RailS achieves simultaneous load balancing on both sender and receiver sides.

**CCT.** As shown in Fig. 14d, RailS achieves the best CCT performance. Compared with other schemes, RailS reduces completion time by more than 50%, benefiting from balanced NIC utilization on the receiver side, which other schemes fail to achieve.

In summary, although RailS also employs a spraying mechanism, its uniform sending pattern naturally induces uniform receiving distribution, making it significantly superior to REPS. Since the bottleneck lies on the single receiver side rather than within the network, PLB and MinRTT even

perform worse than ECMP, as ECMP avoids unnecessary path reorganization in this case.

### F. Real-Trace Workload Performance

We replayed the training and communication emulation of the Mixtral  $8 \times 7B$  MoE training trace on mininet consisting of four servers, connected via two spines and eight leaves with a 2:1 oversubscription ratio. Each server is equipped with two GPUs and eight NICs. The training configuration was set with a sequence length of 2,048, a micro-batch size of 1, and parallelism parameters of DP=1, TP=1, PP=1, and EP=8. Consequently, the 8 experts were distributed with a strict one-to-one mapping to the 8 GPUs (two experts per server). The token input distribution across GPUs was strictly uniform, while the traffic sparsity and imbalance were dynamically generated and controlled by a Top-K gating mechanism with  $K = 2$ .

To evaluate the impact of multi-NIC utilization on all-to-all communication, we distributed the data transmitted by each expert across the available NICs in different ways. In the **dense** setup, each expert was statically assigned 4 NICs for parallel data exchange. In the **sparse** setup, to simulate extreme bottlenecks, each expert was restricted to using only 1 NIC for transmission. In contrast, RailS bypasses static limitations, allowing all experts on a server to dynamically aggregate traffic and utilize all available NICs, thereby fully exploiting the physical multi-rail parallelism.

During training, four phases of the iteration process were identified: **Start** denotes the first iteration, **Early** represents the early stage, **Mid** refers to the middle stage, and **Stable** indicates the stable stage. The input distribution on each GPU remained uniform, while the total data volume increased as the iteration progressed. For example, in the Start phase, the total

TABLE II  
MoE MODEL CONFIGURATIONS

| Model            | Layers | FFN_H | L    | H     | Heads | TP | PP | EP | DP | Experts | Micro_batch | topk |
|------------------|--------|-------|------|-------|-------|----|----|----|----|---------|-------------|------|
| DeepSeek-MoE-16B | 27     | 1408  | 4096 | 10944 | 16    | 1  | 1  | 16 | 1  | 64      | 1           | 8    |
| Qwen3-MoE-235B   | 94     | 1536  | 4096 | 4096  | 64    | 1  | 1  | 16 | 1  | 128     | 10          | 2    |

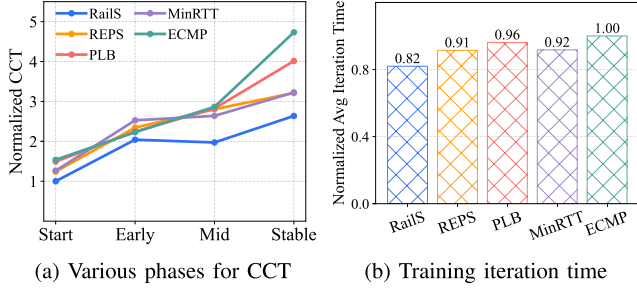


Fig. 15. The Mixtral  $8 \times 7B$  model under dense mode exhibits the following metrics: (a) all-to-all completion time and (b) training iteration time.

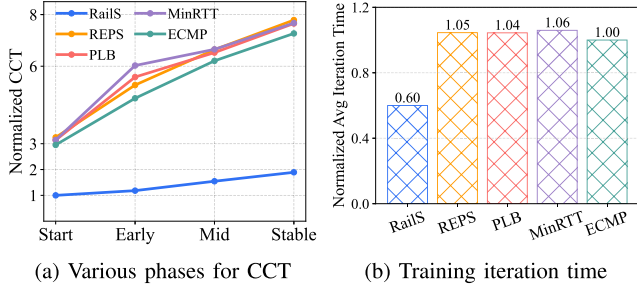


Fig. 16. The Mixtral  $8 \times 7B$  model under sparse mode exhibits the following metrics: (a) all-to-all completion time and (b) training iteration time.

transmission volume per expert was around 100 MB, whereas in the Stable phase, it reached 256 MB.

**Dense Setup.** In Fig. 15, the normalized CCT analysis indicates that RailS reduces the CCT by 19.5%–34.9% in the Start phase, 8.9%–19.2% in the Early phase, 25.3%–31.2% in the Mid phase, and 17.9%–44.3% in the Stable phase. Overall, RailS shortens the iteration time by up to 18% compared with other schemes.

**Sparse Setup.** In Fig. 16, the normalized CCT shows that RailS reduces the CCT by 66.1%–69.2% in the Start phase, 75.1%–80.4% in the Early phase, 75.1%–76.8% in the Mid phase, and 73.9%–75.7% in the Stable phase. Throughout the entire training process, RailS achieves at least a 40% reduction in iteration time compared with other schemes.

These results demonstrate that the sparser the load matrix, the better the performance of the RailS scheme. This is because, under sparse workloads, bottlenecks tend to occur at the receiver side of the Rail architecture rather than within the network. Although some schemes can exploit multipath transmission, they fail to leverage parallel reception, thereby creating receiver-side bottlenecks and performing worse than ECMP. The uniform-sending and uniform-receiving characteristics of RailS fully utilize architectural parallelism and ensure NIC load balancing, achieving optimal performance.

## VII. PROTOTYPE TESTBED

### A. Settings

**Testbed Setup.** We utilize a physical cluster of four servers connected via a two layer leaf and spine topology consisting

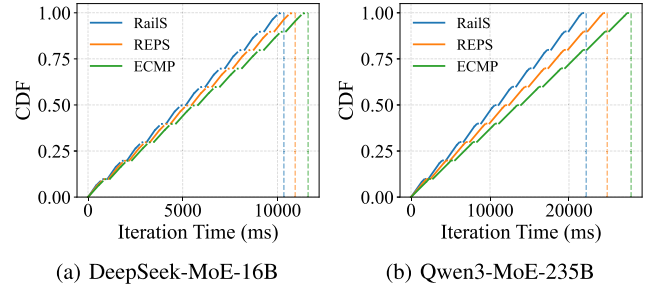


Fig. 17. The iteration completion time performance of the (a) DeepSeek-MoE-16B and (b) Qwen3-MoE-235B models over 10 consecutive runs.

of four spine switches and two leaf switches which all belong to the Cisco 9500 series. To accommodate traffic bursts while maintaining low queuing delay we set the per queue buffer size to 500 KB and configure the ECN thresholds to  $K_{\min} = 30$  KB and  $K_{\max} = 65$  KB. We implement a 2 to 1 oversubscription ratio at the leaf tier to simulate realistic congestion scenarios. Each server is equipped with two Mellanox ConnectX-5 dual port 100 Gbps RDMA NICs where each NIC connects to a leaf switch and the inter switch link rate operates at 100 Gbps.

**MoE Model.** We used real hardware and the simai framework to emulate real MoE traffic. We primarily tested the DeepSeek-MoE-16B [39] and Qwen3-MoE-235B [62] models, with 64 and 128 experts respectively. TP, DP, and PP were all set to 1, and EP was set to 16. The DeepSeek-MoE-16B model used  $topk = 8$ , while the Qwen3-MoE-235B used  $topk = 2$ . More specific layer details of the models can be found in Table II. For the computation component, we leverage the operator-level computation profiling capabilities provided by the SimAI framework. We conduct computation measurements on a 16-GPU cluster consisting of two nodes, each equipped with eight NVIDIA H20 GPUs with 80 GB of memory per GPU. The measurements are performed on the two aforementioned models to align the simulation with real hardware behavior.

### B. Performance

We evaluate the training iteration performance of RailS under realistic MoE workloads. Fig. 17 illustrates the CDF of event completion times across 10 consecutive training iterations, encompassing forward propagation, backward propagation, and parameter optimization.

Across both evaluated models, RailS consistently outperforms the baseline transport schemes. For the DeepSeek-MoE-16B workload in Fig. 17a, RailS achieves an average iteration time of 1,033.7 ms. This represents an average 12.3% reduction compared to ECMP (1,161.6 ms) and a 5.8% improvement over REPS (1,093.4 ms). The performance gap widens under the heavier Qwen3-MoE-235B workload in Fig. 17b, where larger message sizes exacerbate network contention. Here, RailS reduces the average iteration time to 2,218.0 ms, outperforming ECMP (2,786.3 ms) by 25.5% and REPS (2,484.1 ms) by 12.0%.



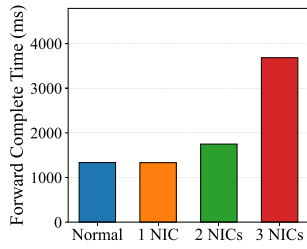


Fig. 18. Fault tolerance.

The performance disparity stems from traffic distribution granularity and topology awareness. ECMP performs the worst as its coarse-grained flow hashing causes collisions and significant load imbalance under bursty MoE traffic. While REPS mitigates these collisions via packet-level spraying, it remains topology-agnostic. RailS achieves the best performance by explicitly coordinating traffic across parallel NICs to leverage the cluster's multi-rail structure. This topology-aware scheduling prevents local contention at the leaf tier while fully utilizing inter-switch links, effectively absorbing MoE microbursts to minimize iteration time.

### C. Fault Tolerance Analysis

We further evaluate the robustness of RailS by simulating NIC port failures on a single server during the Qwen3-MoE forward propagation as shown in Fig. 18. RailS exhibits high resilience to single-link failures. When one NIC port fails, the iteration time increases by a negligible 0.2% from 1,331.1 ms to 1,334.4 ms. This is because RailS effectively leverages the remaining three healthy rails to absorb the sparse MoE traffic without saturating the link bandwidth. Performance degrades gracefully as failures increase; losing two ports results in a 31.4% slowdown to 1,748.6 ms due to halved bandwidth capacity. In the extreme case of three port failures, the system suffers a  $2.8\times$  slowdown to 3,683.4 ms, indicating that while RailS handles partial failures robustly, maintaining sufficient path diversity is essential for peak performance.

### D. Scheduling Overhead Analysis

To justify the design choice of the LPT approximation over an exact solution, we evaluate the computational overhead of the scheduling algorithms. Fig. 18 compares the decision latency of the exact ILP solver (implemented via Gurobi) against our proposed LPT scheduler. The results highlight a critical performance divergence. The LPT algorithm operates with negligible latency, consistently generating scheduling decisions in approximately  $2\text{--}3\ \mu\text{s}$ . In contrast, the ILP solver exhibits high variance and prohibitive overhead, fluctuating between tens of milliseconds to over 200 ms. Given that a typical MoE all-to-all communication phase lasts only tens to hundreds of milliseconds, utilizing an ILP solver would introduce an intolerable overhead, effectively doubling the communication latency. This empirical evidence validates the necessity of our theoretical transition from the exact ILP formulation to the lightweight LPT approximation, ensuring that the load-balancing mechanism itself does not become a system bottleneck.

## VIII. DISCUSSION

In this section, we will discuss various aspects of RailS.

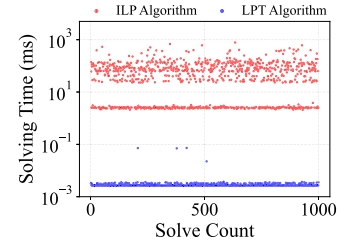


Fig. 19. Scheduling overhead.

**What is the limitation of RailS?** RailS effectively mitigates communication bottlenecks in distributed MoE training by transforming complex global coordination into a local, polynomial-time scheduling task, enabling near-optimal load balancing and multi-NIC parallelism without hardware upgrades. However, its benefits depend on Rail-optimized topologies and high intra-node bandwidth, limiting effectiveness in traditional Fat-Tree networks or systems with weaker interconnects. As a host-based scheduler, RailS also trades off load-balancing accuracy for CPU overhead, making it less suitable for workloads with extremely small messages.

**How do the simulation parameters relate to the actual system, and what is the expected deviation?** Our simulation effectively captures the fundamental characteristics of real-world multi-NIC Rail systems by accurately modeling the network topology, oversubscription ratios, base RTT, and queuing models. However, expected deviations arise primarily from the limitations of the software-based network stack. First, due to the CPU-bound processing limits of Open vSwitch, the absolute link bandwidth in the simulation cannot reach the ultra-high speeds of modern physical hardware, requiring us to scale down the bandwidth to maintain simulation stability. Second, while Soft-RoCE faithfully emulates the protocol logic of RDMA, it cannot fully replicate the hardware-offloaded capabilities of physical NICs, such as zero-copy transmission and kernel bypass. This limitation introduces additional software processing overhead and slight latency variance that would not exist in a pure hardware RDMA environment.

**How does RailS perform under heterogeneous environments and partial failures?** In practical deployments, systems may encounter heterogeneous NIC capabilities, bandwidth imbalances, asymmetric routing, and partial failures. For heterogeneous NICs and bandwidth imbalances, the differences manifest primarily as varying link transmission rates. RailS maintains robustness by dynamically scaling the traffic distribution to be directly proportional to the actual transmission rate of each NIC, thereby aligning the completion times across all paths. Regarding asymmetric routing, the system remains effective because the sender-side scheduling principle decouples local traffic injection from downstream routing variations. Furthermore, as the test platform experiments showed, losing a network card port only results in a slight increase in iteration time. The applicability boundary of RailS is reached only when severe partial failures destroy path diversity and negate the multi-path advantage.

**What are the deployment requirements of RailS?** RailS is designed for high deployability and requires no specialized hardware, firmware modifications, switch OS upgrades, or custom protocol extensions. It operates entirely as a software-based, host-level scheduler using standard RDMA/RoCEv2 protocols. Architecturally, RailS functions as



a lightweight transport-layer plugin or shim layer that sits between existing collective communication libraries and the NIC drivers. The LPT scheduler performs pre-transmission local planning to determine the optimal mapping of atomic flows to specific NICs, and then utilizes standard Queue Pair selection APIs to enforce these routing decisions. Consequently, RailS seamlessly coexists with current AI training stacks, making it highly compatible and easily deployable in standard production data centers without disrupting the established software ecosystem.

## IX. RELATED WORK

The core objective of load balancing is to efficiently distribute network traffic across multiple paths to enhance system throughput and resource utilization. Existing schemes exhibit significant differences in traffic granularity. Traffic granularity can range from individual packets [8], [21], [32], [69] to flowlets [5], [28], [58], [60], to subflows [37], [40], [45], [50] within a single connection, and up to entire connections or flows [4], [24], [68]. The choice of granularity impacts scheduling accuracy and load balancing, with fine-grained partitioning improving allocation precision and coarse-grained partitioning reducing overhead.

Local load balancing distributes traffic across equivalent paths and can be implemented at the host or switch level. Host-level schemes [30], [48], [51], [65] offer greater flexibility, easier scaling, and faster adaptation to network changes, supporting multipath scheduling and improving system performance. Switch-level [6], [7], [10], [54] schemes react quickly to transient congestion and reduce host overhead but provide limited flexibility for policy customization and scaling.

Traffic forwarding decisions in load balancing rely on network state information. Many schemes utilize connectivity information to identify currently available equivalent paths. Significant differences exist in how load information is applied. Some schemes perform static allocation without relying on load information [12], [23], [44]. Others dynamically adjust traffic distribution based on local load conditions to optimize link utilization [29], [56], [64]. Certain schemes employ global load information for unified scheduling to achieve overall performance optimization [31], [46]. Recent works further extend load balancing to reconfigurable data center networks, addressing dynamic topologies and time-varying link conditions [34], [35], [36]. These differences determine the performance of load balancing schemes in terms of throughput, latency, and network resource utilization.

## X. CONCLUSION

Driven by the sparse, dynamic, and imbalanced communication of MoE models, we propose RailS, a load balancer for MoE all-to-all communication. RailS exploits the deterministic topology and symmetry of the Rail architecture to resolve path conflicts and structure multi-NIC resource allocation. Theoretically, we prove that sender-side balancing inherently ensures receiver-side balance, reducing global coordination to locally solvable subproblems. System-wise, an LPT-based scheduler efficiently realizes this principle, achieving near-optimal performance through proactive, fine-grained traffic planning. Experiments demonstrate that RailS enhances bus bandwidth, shortens collective communication and training iteration times, and improves overall GPU and network utilization. We hope this work inspires future exploration

of topology-aware load balancing and transport co-design, encouraging broader investigation into efficient and scalable communication for large MoE systems.

## REFERENCES

- [1] *Infiniband Architecture Specification Release 1.2.1 Annex A16: Roce*, InfiniBand Trade Assoc. (IBTA), Beaverton, OR, USA, 2010.
- [2] *Infiniband Architecture Specification Release 1.2.1 Annex A17: Rocev2*, InfiniBand Trade Assoc. (IBTA), Beaverton, OR, USA, 2014.
- [3] M. Al-Fares, A. Loukissas, and A. Vahdat, "A scalable, commodity data center network architecture," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 38, no. 4, pp. 63–74, Oct. 2008.
- [4] M. Al-Fares, S. Radhakrishnan, B. Raghavan, N. Huang, and A. Vahdat, "Hedera: Dynamic flow scheduling for data center networks," in *Proc. 7th USENIX Conf. Networked Syst. Design Implement.*, 2010, pp. 89–92.
- [5] M. Alizadeh et al., "CONGA: Distributed congestion-aware load balancing for datacenters," in *Proc. ACM Conf. SIGCOMM*, Aug. 2014, pp. 503–514.
- [6] M. Alizadeh et al., "Pfabric: Minimal near-optimal datacenter transport," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 43, no. 4, pp. 435–446, 2013.
- [7] T. Benson, A. Anand, A. Akella, and M. Zhang, "MicroTE: Fine grained traffic engineering for data centers," in *Proc. 7th Conf. Emerg. Netw. Exp. Technol.*, Dec. 2011, pp. 1–12.
- [8] T. Bonato et al., "REPS: Recycled entropy packet spraying for adaptive load balancing and failure mitigation," 2024, *arXiv:2407.21625*.
- [9] T. B. Brown et al., "Language models are few-shot learners," in *Proc. NIPS*, 2020, pp. 1877–1901.
- [10] Q. Cai, M. T. Arashloo, and R. Agarwal, "DePIM: Near-optimal proactive datacenter transport," in *Proc. ACM SIGCOMM Conf.*, Aug. 2022, pp. 53–65.
- [11] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics, Hum. Lang. Technol.*, vol. 1, Jun. 2019, pp. 4171–4186.
- [12] A. Dixit, P. Prakash, Y. C. Hu, and R. R. Kompella, "On the impact of packet spraying in data center networks," in *Proc. IEEE INFOCOM*, Apr. 2013, pp. 2130–2138.
- [13] N. Du et al., "GLaM: Efficient scaling of language models with mixture-of-experts," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 5547–5569.
- [14] W. Fedus, B. Zoph, and N. Shazeer, "Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity," *J. Mach. Learn. Res.*, vol. 23, no. 120, pp. 1–39, 2021.
- [15] A. Gangidi et al., "RDMA over Ethernet for distributed training at meta scale," in *Proc. ACM SIGCOMM Conf.*, Aug. 2024, pp. 57–70.
- [16] R. L. Graham, "Bounds on multiprocessing timing anomalies," *SIAM J. Appl. Math.*, vol. 17, no. 2, pp. 416–429, Mar. 1969.
- [17] A. Greenberg et al., "VL2: A scalable and flexible data center network," in *Proc. ACM SIGCOMM Conf. Data Commun.*, Aug. 2009, pp. 51–62.
- [18] C. Guo et al., "BCube: A high performance, server-centric network architecture for modular data centers," in *Proc. ACM SIGCOMM Conf. Data Commun.*, New York, NY, USA, Aug. 2009, pp. 63–74.
- [19] C. Guo, H. Wu, K. Tan, L. Shi, Y. Zhang, and S. Lu, "Dcell: A scalable and fault-tolerant network structure for data centers," in *Proc. ACM SIGCOMM Conf. Data Commun.*, New York, NY, USA, Aug. 2008, pp. 75–86.
- [20] D. Guo et al., "DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning," *Nature*, vol. 645, no. 8081, pp. 633–638, 2025.
- [21] M. Handley et al., "Re-architecting datacenter networks and stacks for low latency and high performance," in *Proc. Conf. ACM Special Interest Group Data Commun.*, Aug. 2017, pp. 29–42.
- [22] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.
- [23] K. He et al., "Presto: Edge-based load balancing for fast datacenter networks," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 45, no. 4, pp. 465–478, 2015.
- [24] C. Hopps, "Analysis of an equal-cost multi-path algorithm," Internet Eng. Task Force (IETF), USA, Tech. Rep. RFC 2992, 2000.
- [25] J. Hu et al., "Load balancing with multi-level signals for loss-less datacenter networks," *IEEE/ACM Trans. Netw.*, vol. 32, no. 3, pp. 2736–2748, Jun. 2024.
- [26] S. Hu et al., "Deadlocks in datacenter networks: Why do they form, and how to avoid them," in *Proc. 15th ACM Workshop Hot Topics Netw.*, 2016, pp. 92–98.

- [27] A. Q. Jiang et al., “Mixtral of experts,” 2024, *arXiv:2401.04088*.
- [28] A. Kabbani, B. Vamanan, J. Hasan, and F. Duchene, “FlowBender: Flow-level adaptive routing for improved latency and throughput in datacenter networks,” in *Proc. 10th ACM Int. Conf. Emerg. Netw. Experiments Technol.*, Dec. 2014, pp. 149–160.
- [29] S. Kandula, D. Katabi, S. Sinha, and A. Berger, “Dynamic load balancing without packet reordering,” *ACM SIGCOMM Comput. Commun. Rev.*, vol. 37, no. 2, pp. 51–62, Mar. 2007.
- [30] N. Katta et al., “Clove: Congestion-aware load balancing at the virtual edge,” in *Proc. 13th Int. Conf. Emerg. Netw. Experiments Technol.*, Nov. 2017, pp. 323–335.
- [31] N. Katta, M. Hira, C. Kim, A. Sivaraman, and J. Rexford, “HULA: Scalable load balancing using programmable data planes,” in *Proc. Symp. SDN Res. (SOSR)*, 2016, pp. 1–12.
- [32] Y. Le et al., “STrack: A reliable multipath transport for AI/ML clusters,” 2024, *arXiv:2407.15266*.
- [33] D. Lepikhin et al., “GShard: Scaling giant models with conditional computation and automatic sharding,” 2020, *arXiv:2006.16668*.
- [34] J. Li, F. De Marchi, Y. Lei, R. Joshi, B. Chandrasekaran, and Y. Xia, “Unlocking diversity of fast-switched optical data center networks with unified routing,” *IEEE Trans. Netw.*, vol. 34, pp. 33–48, 2026.
- [35] J. Li et al., “Uniform-cost multi-path routing for reconfigurable data center networks,” in *Proc. ACM SIGCOMM Conf.*, Aug. 2024, pp. 433–448.
- [36] J. Li, Y. Lei, F. De Marchi, R. Joshi, B. Chandrasekaran, and Y. Xia, “Hop-on hop-off routing: A fast tour across the optical data center network for latency-sensitive flows,” in *Proc. 6th Asia Pacific Workshop Netw.*, Jul. 2022, pp. 63–69.
- [37] J. Li, S. Tripathi, L. Rastogi, Y. Lei, R. Pan, and Y. Xia, “Optimizing mixture-of-experts inference time combining model deployment and communication scheduling,” 2024, *arXiv:2410.17043*.
- [38] X. Liao et al., “MixNet: A runtime reconfigurable optical-electrical fabric for distributed mixture-of-experts training,” in *Proc. ACM SIGCOMM Conf.*, Sep. 2025, pp. 554–574.
- [39] A. Liu et al., “DeepSeek-V3 technical report,” 2024, *arXiv:2412.19437*.
- [40] Y. Lu, “Multi-path transport for RDMA in datacenters,” in *Proc. 15th USENIX Symp. Networked Syst. Design Implement.*, 2018, pp. 357–371.
- [41] R. Mittal et al., “Revisiting network support for RDMA,” in *Proc. Conf. ACM Special Interest Group Data Commun.*, Aug. 2018, pp. 313–326.
- [42] M. Mitzenmacher, “The power of two choices in randomized load balancing,” *IEEE Trans. Parallel Distrib. Syst.*, vol. 12, no. 10, pp. 1094–1104, 2002.
- [43] DGX NVIDIA.(2023). *SuperPOD: Next Generation Scalable Infrastructure for AI Leadership*. [Online]. Available: <https://docs.nvidia.com/https://docs.nvidia.com/dgx-superpod-reference-architecture-dgx-h100.pdf>
- [44] V. Olteanu et al., “An edge-queued datagram service for all datacenter traffic,” in *Proc. 19th USENIX Symp. Networked Syst. Design Implement. (NSDI 22)*, 2022, pp. 761–777.
- [45] C. Paasch, S. Ferlin, O. Alay, and O. Bonaventure, “Experimental evaluation of multipath TCP schedulers,” in *Proc. ACM SIGCOMM Workshop Capacity Sharing Workshop*, Aug. 2014, pp. 27–32.
- [46] J. Perry et al., “Fastpass: A centralized,” in *Proc. ACM Conf. SIGCOMM*, 2014, pp. 307–318.
- [47] K. Qian et al., “Alibaba HPN: A data center network for large language model training,” in *Proc. ACM SIGCOMM Conf.*, Aug. 2024, pp. 691–706.
- [48] M. A. Qureshi et al., “PLB: Congestion signals are simple and effective for network load balancing,” in *Proc. ACM SIGCOMM Conf.*, Aug. 2022, pp. 207–218.
- [49] C. Raiciu, S. Barre, C. Pluntke, A. Greenhalgh, D. Wischik, and M. Handley, “Improving datacenter performance and robustness with multipath TCP,” *ACM SIGCOMM Comput. Commun. Rev.*, vol. 41, no. 4, pp. 266–277, Oct. 2011.
- [50] C. Raiciu, D. Niculescu, M. Bagnulo, and M. J. Handley, “Opportunistic mobility with multipath TCP,” in *Proc. 6th Int. workshop MobiArch*, Jun. 2011, pp. 7–12.
- [51] C. Raiciu et al., “How hard can it be? Designing and implementing a deployable multipath TCP,” in *Proc. 9th USENIX Symp. Networked Syst. Design Implement. (NSDI)*, 2012, pp. 399–412.
- [52] S. Rajbhandari et al., “DeepSpeed-MoE: Advancing mixture-of-experts inference and training to power next-generation AI scale,” in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 18332–18346.
- [53] R. Recio, “A remote direct memory access protocol specification,” Internet Eng. Task Force (IETF), USA, Tech. Rep. RFC 5040, 2007.
- [54] S. Sen, D. Shue, S. Ihm, and M. J. Freedman, “Scalable, optimal flow routing in datacenters via local link balancing,” in *Proc. 9th ACM Conf. Emerg. Netw. Exp. Technol.*, 2013, pp. 151–162.
- [55] A. Singh et al., “Jupiter rising: A decade of clos topologies and centralized control in Google’s datacenter network,” in *Proc. ACM Conf. Special Interest Group Data Commun.*, Aug. 2015, pp. 183–197.
- [56] C. H. Song, “Network load balancing with in-network reordering support for RDMA,” in *Proc. ACM SIGCOMM 2023 Conf.*, 2023, pp. 816–831.
- [57] H. Touvron et al., “LLaMA: Open and efficient foundation language models,” 2023, *arXiv:2302.13971*.
- [58] E. Vanini, R. Pan, M. Alizadeh, P. Taheri, and T. Edsall, “Let it flow: Resilient asymmetric load balancing with flowlet switching,” in *Proc. 14th USENIX Symp. Netw. Syst. Design Implement. (NSDI)*, 2017, pp. 407–420.
- [59] Y. Wan, H. Song, Y. Jia, Y. Yang, T. Huang, and Z. Chen, “LAPS: Joint load balancing and congestion control on unequal-cost multi-path data center networks,” in *Proc. 2nd Workshop Netw. AI Comput.*, Sep. 2025, pp. 11–18.
- [60] P. Wang, H. Xu, Z. Niu, D. Han, and Y. Xiong, “Expeditus: Congestion-aware load balancing in clos data center networks,” in *Proc. 7th ACM Symp. Cloud Comput.*, Oct. 2016, pp. 442–455.
- [61] X. Wang et al., “SimAI: Unifying architecture design and performance tuning for large-scale large language model training with scalability and precision,” in *Proc. 22nd USENIX Symp. Networked Syst. Design Implement.*, 2025, pp. 541–558.
- [62] A. Yang et al., “Qwen3 technical report,” 2025, *arXiv:2505.09388*.
- [63] J. Y. Yen, “Finding the K shortest loopless paths in a network,” *Manage. Sci.*, vol. 17, no. 11, pp. 712–716, Jul. 1971.
- [64] D. Zats, T. Das, P. Mohan, D. Borthakur, and R. Katz, “Detail: Reducing the flow completion time tail in datacenter networks,” in *Proc. ACM SIGCOMM Conf. Appl., Technol., Archit., Protocols Comput. Commun.*, 2012, pp. 139–150.
- [65] H. Zhang, J. Zhang, W. Bai, K. Chen, and M. Chowdhury, “Resilient datacenter load balancing in the wild,” in *Proc. Conf. ACM Special Interest Group Data Commun.*, Aug. 2017, pp. 253–266.
- [66] J. Zhang, W. Bai, and K. Chen, “Enabling ECN for datacenter networks with RTT variations,” in *Proc. 15th Int. Conf. Emerg. Netw. Exp. Technol.*, Dec. 2019, pp. 233–245.
- [67] C. Zhao et al. (2025). *DeepEP: An Efficient Expert-Parallel Communication Library*. [Online]. Available: <https://github.com/deepseek-ai/DeepEP>
- [68] J. Zhou et al., “WCMP: Weighted cost multipathing for improved fairness in data centers,” in *Proc. 9th Eur. Conf. Comput. Syst.*, Apr. 2014, pp. 1–14.
- [69] Y. Zhou et al., “An extensible software transport layer for GPU networking,” 2025, *arXiv:2504.17307*.